

Bàn về ứng dụng của ngôn ngữ C++ trong thi tin học

Han Wentao

Trường Trung học Dư Diêu, Chiết Giang

IOI 2004, đội tuyển tập huấn quốc gia Trung Quốc

Nguồn gốc: Bàn về ứng dụng của ngôn ngữ C++ trong thi tin học

IOI China candidate team papers / 2004 / Han Wentao.pdf

Abstract

Ngôn ngữ lập trình là một bộ phận quan trọng của thi tin học: mọi thuật toán chỉ thật sự giải được bài toán sau khi được hiện thực bằng một ngôn ngữ lập trình. Nhờ tính linh hoạt cao và hệ thống thư viện mạnh, C++ được dùng rất rộng rãi trong các cuộc thi sinh viên, và cũng ngày càng phổ biến hơn trong các cuộc thi dành cho học sinh trung học. Bài viết gồm ba chương, lần lượt giới thiệu từ nông đến sâu những kiến thức cơ bản của C++, nền tảng lập trình hướng đối tượng cùng một số thư viện chuẩn thường dùng, và Thư viện Khuôn mẫu Chuẩn (STL). Hy vọng bài viết có ích cho những độc giả muốn dùng C++ trong thi tin học.

Từ khóa: thi tin học; ngôn ngữ lập trình C++; Thư viện Khuôn mẫu Chuẩn.

Contents

Lời nói đầu	1
1 Từ Pascal đến C++	2
1.1 Thế giới, xin chào!	2
1.2 Kiểu và định nghĩa	2
1.3 Con trỏ, mảng và cấu trúc	6
1.4 Biểu thức và câu lệnh	9
1.5 Hàm	12
1.6 Một số hàm thư viện thường dùng	14
1.7 Kết chương	15
2 Đi sâu hơn vào ngôn ngữ C++	17
2.1 Lớp	17
2.2 Nạp chồng toán tử	22
2.3 Chuỗi	23
2.4 Luồng	24
2.5 Kết chương	27

3	Giới thiệu STL	29
3.1	Khái quát STL	29
3.2	Bộ lặp	30
3.3	Thuật toán	31
3.4	Vật chứa	38
3.5	Kết chương	41
	Tổng kết	42
	Tài liệu tham khảo	43

Lời nói đầu

Thi tin học thường yêu cầu thí sinh trong một thời gian nhất định phải hiểu và phân tích đề, thiết kế thuật toán thỏa mãn giới hạn thời gian và bộ nhớ, rồi dùng một ngôn ngữ lập trình để hiện thực thuật toán đúng đắn trên máy tính. Vì toàn bộ cuộc thi bị giới hạn thời gian, nhất là trong các cuộc thi kiểu ACM/ICPC, nơi số bài bằng nhau thì thứ hạng phụ thuộc vào tổng thời gian làm bài, nên ngôn ngữ lập trình có giúp hiện thực thuật toán đúng và nhanh hay không ảnh hưởng khá lớn đến kết quả. Vì vậy, độ phức tạp khi lập trình ngày càng được coi trọng; nó lại phụ thuộc rất nhiều vào ngôn ngữ được chọn. Một số ngôn ngữ thường gặp trong thi tin học có đặc điểm như sau.

	BASIC	Pascal	C++	Java
Độ khó học	dễ	trung bình	khó hơn	khó hơn
Đặc điểm ngôn ngữ	đơn giản	chặt chẽ	linh hoạt	hướng đối tượng cao
Tốc độ chạy	chậm	khá nhanh	nhanh	chậm
Chức năng thư viện	yếu	trung bình	rất mạnh	mạnh

Trong các cuộc thi tin học học sinh trung học ở thời điểm bài viết, Pascal được dùng khá rộng rãi. Nhưng C++, nhờ sự linh hoạt vốn có và thư viện rất mạnh, đang được ngày càng nhiều thí sinh sử dụng. Bài viết này ra đời trong bối cảnh đó. Trong bài viết, Pascal được hiểu theo Free Pascal, vì chuẩn Pascal cung cấp quá ít và Free Pascal phổ biến hơn trong thi đấu; C++ được hiểu theo chuẩn ANSI/ISO C++.

Cần chú ý rằng đây không phải là tài liệu tra cứu. Vì vậy, bài viết không thể giới thiệu mọi mặt của C++ với đầy đủ chi tiết. Muốn học sâu hơn về C++, độc giả nên đọc thêm các tài liệu chuyên biệt.

1 Từ Pascal đến C++

Điều kiện cần để đọc chương này là đã biết dùng Pascal và có một lượng kiến thức thuật toán nhất định. Vì hiện nay trong thi tin học trung học vẫn có nhiều thí sinh dùng Pascal, mục đích của chương này là giúp độc giả đã nắm Pascal có thể chuyển sang C++ càng nhanh càng tốt. Nếu đã quen với C++, có thể bỏ qua chương này và đọc thẳng chương sau.

1.1 Thế giới, xin chào!

Trước hết, hãy dùng một chương trình C++ rất đơn giản để có ấn tượng ban đầu về ngôn ngữ này.

```
//: C01:Hello.cpp
```

```
// Hello, world!
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, world!" << endl;
} ///:~
```

Chương trình này in dòng chữ "Hello, world!" ra màn hình.

Trong dòng thứ nhất, dòng thứ hai và dòng cuối, phần bắt đầu từ // đến hết dòng đều là chú thích. Trong C++ có hai cách viết chú thích. Một cách là chú thích khối kế thừa từ C, bắt đầu bằng /* và kết thúc bằng */. Cách còn lại là chú thích dòng, bắt đầu từ // và kéo dài đến cuối dòng.

Nội dung bắt đầu bằng # ở dòng thứ ba gọi là chỉ thị tiền xử lý. Dòng này đưa tập tin đầu iostream vào chương trình, tương tự câu lệnh uses để dùng unit trong Pascal. Tuy nhiên, Pascal mặc định dùng unit System, trong đó chứa phần lớn thủ tục và hàm thường dùng, còn C++ mặc định không đưa tập tin đầu nào vào. Ngoài ra, tập tin đầu của C thường có đuôi .h, còn C++ chuẩn khuyến khích dùng tập tin đầu không có phần mở rộng.

Dòng thứ tư cho phép ta trực tiếp dùng các định danh trong không gian tên std. Không gian tên std chứa mọi lớp và hàm do C++ chuẩn cung cấp; để tiện, người ta thường viết dòng này ngay sau các chỉ thị đưa tập tin đầu vào. Chi tiết về không gian tên có thể xem thêm tài liệu khác.

Dòng thứ năm đến thứ bảy là thân chương trình. Ta định nghĩa một hàm tên main, tương đương chương trình chính trong Pascal. Mọi chương trình C++ có thể chạy độc lập đều cần có hàm main. Trong C++, hai dấu { và } tương đương begin và end của Pascal.

Dòng thứ sáu in Hello, world! rồi xuống dòng. Ở đây cout biểu diễn đầu ra chuẩn, thường là màn hình, tương tự Output trong Pascal. Toán tử << vốn là phép dịch trái bit, nhưng ở đây được định nghĩa lại thành toán tử chèn, đưa một nội dung vào luồng xuất. endl có nghĩa là xuống dòng. Chuỗi trong C++ được đặt giữa dấu nháy kép, và mỗi câu lệnh kết thúc bằng dấu chấm phẩy.

1.2 Kiểu và định nghĩa

Kiểu

Giống Pascal, C++ cung cấp các kiểu cơ bản và cho phép lập trình viên tự định nghĩa kiểu. Bảng sau liệt kê một số kiểu cơ bản thường dùng trong C++ cùng kiểu Pascal tương ứng.

Tên	Kiểu C++	Kiểu Pascal	Miền giá trị	Byte
Logic	bool	Boolean	true/false	1
Ký tự	char	Char	mọi ký tự một byte	1
Số nguyên có dấu 8 bit	char	ShortInt	-128..127	1
Số nguyên không dấu 8 bit	unsigned char	Byte	0..255	1
Số nguyên có dấu 16 bit	short	SmallInt	-32768..32767	2
Số nguyên không dấu 16 bit	unsigned short	Word	0..65535	2
Số nguyên có dấu 32 bit	int	LongInt	-2147483648..2147483647	4
Số nguyên không dấu 32 bit	unsigned int	LongWord	0..4294967295	4
Số nguyên có dấu 64 bit	long long	Int64	$-2^{63}..2^{63} - 1$	8
Số nguyên không dấu 64 bit	unsigned long long	QWord	$0..2^{64} - 1$	8
Dấu phẩy động đơn	float	Single	1.17e-38..3.40e38	4
Dấu phẩy động đôi	double	Double	2.22e-308..1.79e308	8
Dấu phẩy động mở rộng	long double	Extended	3.36e-4932..1.18e4932	10/12

Kiểu logic. Giống Pascal, kiểu logic biểu diễn kết quả của các phép toán logic. Một giá trị kiểu logic chỉ có thể là `true` hoặc `false`; giá trị số của `true` là 1, còn của `false` là 0. Khác Pascal, trong C++ nhiều kiểu khác có thể được chuyển ngầm sang kiểu logic: giá trị khác không trở thành `true`, còn không trở thành `false`.

Kiểu ký tự. Hằng ký tự đơn được đặt trong dấu nháy đơn. Một số ký tự không hiển thị có thể được biểu diễn bằng chuỗi thoát. Ngoài ra, từ bảng trên có thể thấy trong C++, kiểu ký tự và kiểu nguyên một byte thực chất tương đương; chẳng hạn giá trị số của `'A'` là 65.

Tên	Tên ASCII	Tên trong C++
Xuống dòng	NL/LF	<code>\n</code>
Tab ngang	HT	<code>\t</code>
Tab dọc	VT	<code>\v</code>
Lùi một ký tự	BS	<code>\b</code>
Về đầu dòng	CR	<code>\r</code>
Sang trang	FF	<code>\f</code>
Chuông	BEL	<code>\a</code>
Dấu gạch chéo ngược	<code>\</code>	<code>\\</code>
Dấu hỏi	<code>?</code>	<code>\?</code>
Dấu nháy đơn	<code>'</code>	<code>\'</code>
Dấu nháy kép	<code>"</code>	<code>\"</code>
Mã ASCII bát phân	ooo	<code>\ooo</code>
Mã ASCII thập lục phân	hhh	<code>\xhhh</code>

Kiểu nguyên. Kiểu nguyên trong C++ gần giống Pascal. Đáng chú ý là tên kiểu nguyên C++ thực ra gồm ba phần. Thứ nhất là `signed` hoặc `unsigned`, chỉ có dấu hay không dấu, mặc định là có dấu. Thứ hai là từ bỏ nghĩa độ dài trừ `char`, gồm `short`, `long`, `long long`, mặc định là `long`. Cuối cùng là `int`, có thể bỏ qua nếu phía trước đã có từ bỏ nghĩa.

Hằng nguyên trong C++ gồm tiền tố, dãy chữ số và hậu tố. Tiền tố mô tả cơ số: mặc định là thập phân, bắt đầu bằng 0 là bát phân, bắt đầu bằng 0x là thập lục phân. Hậu tố mô tả độ dài và tính không dấu: `l` là số nguyên 32 bit, `ll` là số nguyên 64 bit, `u` là không dấu, và có thể kết hợp các hậu tố này. Chữ cái trong tiền tố và hậu tố không phân biệt hoa thường.

Kiểu dấu phẩy động. Kiểu dấu phẩy động trong C++ cũng gần giống Pascal. Hằng dấu phẩy động có hai cách viết: dạng cố định và dạng khoa học. Dạng cố định cơ bản giống Pascal, khác ở chỗ trong C++ nếu phần nguyên hoặc phần thập phân bằng không thì có thể bỏ qua phần đó, nhưng dấu chấm thập phân phải giữ lại; ví dụ `39.` và `.142857` đều hợp lệ. Dạng khoa học dùng `e` để tách định trị và số mũ. Hằng dấu phẩy động trong C++ còn có thể mang hậu tố: `f` biểu diễn kiểu đơn, `l` biểu diễn kiểu mở rộng, không có hậu tố thì là kiểu đôi.

Kiểu liệt kê. C++ cũng có kiểu liệt kê tương tự Pascal:

```
enum keyword {ASM, AUTO, BREAK}
```

Kiểu liệt kê này có tên `keyword`, giá trị có thể là `ASM`, `AUTO` hoặc `BREAK`.

C++ không cung cấp sẵn kiểu miền con và tập hợp. Nó cũng không cung cấp kiểu chuỗi tích hợp, nhưng có hai cách xử lý chuỗi. Một cách kế thừa từ C, dựa trên con trỏ ký tự và khá phiền; chi tiết có thể

xem tài liệu khác. Cách còn lại là lớp `string` của thư viện chuẩn C++, thuận tiện hơn và sẽ được trình bày ở mục 2.3.

Để biết mỗi kiểu thực tế chiếm bao nhiêu byte, hãy xem chương trình sau.

```
//: C01:Types.cpp
// Built-in types
#include <iostream>
using namespace std;
int main() {
    cout << "Size of bool: "
          << sizeof(bool) << endl;
    cout << "Size of char: "
          << sizeof(char) << endl;
    cout << "Size of unsigned char: "
          << sizeof(unsigned char) << endl;
    cout << "Size of short: "
          << sizeof(short) << endl;
    cout << "Size of unsigned short: "
          << sizeof(unsigned short) << endl;
    cout << "Size of int: "
          << sizeof(int) << endl;
    cout << "Size of unsigned int: "
          << sizeof(unsigned int) << endl;
    cout << "Size of long long: "
          << sizeof(long long) << endl;
    cout << "Size of unsigned long long: "
          << sizeof(unsigned long long) << endl;
    cout << "Size of float: "
          << sizeof(float) << endl;
    cout << "Size of double: "
          << sizeof(double) << endl;
    cout << "Size of long double: "
          << sizeof(long double) << endl;
} ///:~
```

Ở đây `sizeof` trả về kích thước của kiểu trong ngoặc, và cũng có thể áp dụng cho biến hoặc hằng.

Định nghĩa

Khác Pascal, định nghĩa trong C++ gần như có thể xuất hiện ở mọi nơi trong chương trình. Một định nghĩa gồm bốn phần: từ bỏ nghĩa tùy chọn, kiểu cơ bản, thân định nghĩa và giá trị đầu tùy chọn, thường kết thúc bằng dấu chấm phẩy. Từ bỏ nghĩa thường chỉ định những nội dung không phụ thuộc kiểu. Thân định nghĩa gồm một tên và một số toán tử bỏ nghĩa; các toán tử bỏ nghĩa này sẽ được nói kỹ hơn ở mục 1.3. Ví dụ:

```
int number;
char c = 'H';
const double pi = 3.141592653579893;
```

Ba dòng trên lần lượt định nghĩa biến nguyên number, biến ký tự c được khởi tạo bằng 'H', và hằng kiểu double pi được khởi tạo bằng 3.141592653579893. Các biến cùng kiểu có thể được định nghĩa trong cùng một câu lệnh:

```
int a = 39, b, c = 23;
```

Dòng này định nghĩa ba biến nguyên a, b, c, trong đó a và c lần lượt được khởi tạo bằng 39 và 23.

Với biến không có giá trị đầu, nếu là biến toàn cục, tức được định nghĩa ngoài mọi hàm, nó sẽ được khởi tạo bằng không theo kiểu tương ứng; nếu là biến cục bộ, tức được định nghĩa trong một hàm, nó không được tự động khởi tạo.

Tên hay định danh trong C++ có thể gồm chữ cái và chữ số, nhưng ký tự đầu phải là chữ cái; dấu gạch dưới cũng được xem là một chữ cái. Cần nhớ rằng khác Pascal, C++ phân biệt chữ hoa và chữ thường.

Định nghĩa kiểu

Nếu đặt typedef trước một định nghĩa, định nghĩa ấy trở thành định nghĩa kiểu. Ví dụ:

```
typedef unsigned long long QWord;
```

Khi đó QWord không còn là tên biến, mà là tên kiểu. Điều này tương đương việc làm trong mệnh đề type của Pascal. Sau đó ta có thể dùng tên này để định nghĩa biến mới, thường giúp các định nghĩa phía sau ngắn hơn.

```
QWord x;
```

Câu lệnh này định nghĩa biến x có kiểu QWord, tức unsigned long long.

Vòng đời của định nghĩa

Trong C++, một tên có thể được dùng sau khi đã được định nghĩa. Tuy nhiên tên cũng có một vòng đời, hay phạm vi hiệu lực. Khi vượt ra ngoài phạm vi đó, tên mất hiệu lực và có thể được định nghĩa lại.

Nói chung, vòng đời của biến toàn cục là toàn bộ chương trình. Vòng đời của biến cục bộ bắt đầu từ chỗ định nghĩa đến khi kết thúc khối chương trình chứa định nghĩa đó; khối chương trình là một đoạn mã đặt trong cặp ngoặc nhọn. Khi biến cục bộ trùng tên với biến toàn cục, biến toàn cục bị che khuất và tạm thời không dùng được bằng tên đó.

1.3 Con trỏ, mảng và cấu trúc

Con trỏ

Pascal cũng có con trỏ, nhưng con trỏ trong C++ mạnh hơn nhiều. Với mọi kiểu T, kiểu T* biểu diễn kiểu con trỏ trỏ tới một giá trị kiểu T; dấu * ở đây là toán tử bổ nghĩa kiểu đã nói ở mục trước. Ví dụ:

```
char c = 'a';  
char* p = &c;
```

Biến p là con trỏ tới một giá trị kiểu char và được khởi tạo để trỏ tới biến c. Toán tử & lấy địa chỉ bộ nhớ của biến phía sau nó.

Thao tác cơ bản nhất với con trỏ là lấy nội dung mà nó trỏ tới. Trong C++, thao tác này dùng toán tử *.

```
char c2 = *p;
```

Lúc này giá trị của `c2` là `'a'`. Biến con trỏ cũng có thể được gán bằng 0; khi đó nó không trỏ tới ô nhớ hợp lệ nào. Lấy giá trị từ con trỏ bằng 0 sẽ gây lỗi. Con trỏ trong C++ còn hỗ trợ nhiều phép toán khác, sẽ được mô tả ở mục 1.4.

Cấp phát bộ nhớ động

Giống Pascal, C++ có thể cấp phát bộ nhớ động. Việc cấp phát và thu hồi bộ nhớ được thực hiện bằng toán tử `new` và `delete`, tương tự `New` và `Dispose` trong Pascal, nhưng cách dùng khác nhau.

```
int* p = new int;
int* p2 = new int[10];
delete p;
delete[] p2;
```

Câu lệnh đầu cấp phát không gian cho một biến nguyên và lưu địa chỉ đầu vào `p`. Câu lệnh thứ hai cấp phát không gian liên tục cho 10 biến nguyên và lưu địa chỉ của biến đầu tiên vào `p2`. Hai câu lệnh sau thu hồi không gian đã cấp phát. Chú ý ở câu lệnh thứ tư, sau `delete` có cặp ngoặc vuông rỗng, biểu thị thu hồi không gian của một dãy biến. Khi cấp phát nhiều biến liên tiếp, số lượng có thể là biến nguyên hoặc hằng nguyên.

Con trỏ hằng và con trỏ tới hằng

Vì con trỏ cũng là một kiểu dữ liệu, nên có biến con trỏ và hằng con trỏ. Điểm này hơi phức tạp vì ngoài bản thân con trỏ còn có kiểu cơ sở mà nó trỏ tới. Do đó, hằng liên quan đến con trỏ được chia thành con trỏ hằng và con trỏ tới hằng.

Con trỏ hằng nghĩa là bản thân con trỏ là hằng: địa chỉ nó trỏ tới không thể đổi, nhưng nội dung tại địa chỉ đó vẫn có thể đổi. Nói cách khác, `p` là hằng còn `*p` không phải hằng. Kiểu của con trỏ hằng là `T* const`.

Con trỏ tới hằng nghĩa là bản thân con trỏ không phải hằng: địa chỉ nó trỏ tới có thể đổi, nhưng nội dung mà nó trỏ tới là hằng và không thể đổi. Nói cách khác, `p` không phải hằng còn `*p` là hằng. Kiểu này có thể viết `const T*` hoặc `T const*`.

Hai khái niệm này có thể kết hợp thành con trỏ hằng tới hằng. Khi đó cả `p` và `*p` đều là hằng, với kiểu `const T* const` hoặc `T const* const`.

Mảng

Trong C++, với mọi kiểu `T`, `T[n]` biểu diễn kiểu mảng gồm n phần tử kiểu `T`. Số n phải là hằng xác định được lúc biên dịch.

```
float v[3];
char* a[32];
```

Câu lệnh đầu định nghĩa mảng `v` gồm 3 phần tử kiểu `float`; câu lệnh thứ hai định nghĩa mảng `a` gồm 32 phần tử kiểu con trỏ ký tự. Cần nhớ rằng chỉ số mảng trong C++ đều bắt đầu từ 0. Vì vậy ba phần tử của `v` là `v[0]`, `v[1]` và `v[2]`; mảng `a` cũng tương tự. Điều này có thể làm người quen Pascal thấy không tự nhiên.

Mảng nhiều chiều trong C++ được hiện thực bằng mảng mà phần tử cũng là mảng.

```
int d2[10][20];
```

Câu lệnh này định nghĩa một mảng hai chiều, chiều thứ nhất kích thước 10, chiều thứ hai kích thước 20. Giống Pascal, mảng nhiều chiều trong C++ được lưu theo thứ tự ưu tiên hàng. Đáng chú ý là cả khi định nghĩa và khi truy cập phần tử, mỗi chiều đều phải đặt trong một cặp ngoặc vuông riêng, không dùng dấu phẩy để ngăn cách như trong Pascal. Thú vị hơn, dấu phẩy trong C++ cũng là một toán tử: nó lần lượt tính các biểu thức được ngăn bằng dấu phẩy và trả về giá trị của biểu thức cuối cùng. Vì vậy `d2[1, 2]` không biểu diễn `d2[1][2]`, mà tương đương `d2[2]`, tức một mảng một chiều gồm 20 số nguyên. Độc giả chuyển từ Pascal sang C++ cần tránh lỗi này. Phần lớn trình biên dịch C++ cũng không kiểm tra vượt biên mảng lúc chạy, nên lập trình viên phải tự chú ý.

Khởi tạo mảng

Khởi tạo mảng tương tự khởi tạo biến thường, nhưng mọi phần tử phải đặt trong ngoặc nhọn và ngăn cách bằng dấu phẩy.

```
int v1[] = {1, 2, 3, 4};
char v2[3] = {'a', 'b', 'c'};
int v3[8] = {1, 2, 3, 4};
```

Số trong ngoặc vuông có thể bỏ qua. Nếu bỏ qua, kích thước mảng được quyết định bởi số phần tử trong ngoặc nhọn. Nếu số trong ngoặc vuông lớn hơn số phần tử khởi tạo, các phần tử cuối còn thiếu giá trị sẽ được khởi tạo bằng không. Nếu số trong ngoặc vuông nhỏ hơn số phần tử khởi tạo, trình biên dịch sẽ báo lỗi.

Quan hệ giữa mảng và con trỏ

Trong C++, con trỏ và mảng có liên hệ rất chặt. Tên của một mảng có thể được dùng như con trỏ trỏ tới phần tử đầu tiên, hay phần tử thứ không, của mảng đó.

```
int v[] = {1, 2, 3, 4};
int* p1 = v;
int* p2 = &v[0];
int* p3 = &v[4];
```

Ngược lại, con trỏ cũng có thể được dùng như một mảng.

```
int v[] = {1, 2, 3, 4};
int* p = v;
v[2] = 1;
p[2] = 1;
```

Hai câu lệnh cuối là tương đương.

Tham chiếu

Tham chiếu là bí danh của một biến. Nó chủ yếu được dùng làm tham số hàm và làm kiểu trả về của một số toán tử được nạp chồng. Với mọi kiểu *T*, *T*& biểu diễn kiểu tham chiếu đến *T*.


```
int i = 1;
int& r = i; // r và i biểu diễn cùng một biến nguyên
int x = r; // x = 1
r = 2;      // i = 2
```

Tham chiếu bắt buộc phải được khởi tạo khi định nghĩa, để bảo đảm nó thật sự đại diện cho một vùng nhớ xác định.

Cấu trúc

Kiểu cấu trúc trong C++ tương đương kiểu bản ghi trong Pascal.

```
struct character_type {
    char name[20];
    int level;
    int life;
    int mana;
    int experience;
    int money;
};
```

Đoạn trên định nghĩa kiểu `character_type`. Giống Pascal, biến kiểu bản ghi có thể dùng dấu chấm để truy cập các thành viên.

```
character_type necromancer;
necromancer.level = 99;
necromancer.money = 123456789;
character_type* p = &necromancer;
(*p).life = 1024;
p->mana = 2048;
```

Với con trỏ trỏ tới một giá trị cấu trúc, ngoài cách `(*ptr).field` ta cũng có thể dùng `ptr->field`. Không thể viết `*ptr.field`, vì dấu chấm có độ ưu tiên cao hơn `*`; cách viết đó có nghĩa là `ptr` là một biến cấu trúc có thành viên `field` kiểu con trỏ, và ta muốn dùng nội dung mà `ptr.field` trỏ tới.

Trong C++, định nghĩa danh sách liên kết cũng gọn hơn Pascal.

```
struct node_type {
    data_type data;
    node_type* next;
};
```

Tuy vậy, có lúc không tránh khỏi việc các cấu trúc lồng nhau.

```
struct list; // định nghĩa day du se xuất hiện sau
struct link {
    list* data;
    link* prev;
    link* next;
```

```
};
struct list {
    link* head;
};
```

Khi đó phải khai báo rõ ràng trước để đưa tên một kiểu ra trước. Tình huống này cũng xuất hiện khi các hàm gọi đệ quy lẫn nhau.

1.4 Biểu thức và câu lệnh

Biểu thức

Giống Pascal, biểu thức trong C++ được tạo bởi toán tử và toán hạng.

Toán hạng. Toán hạng trong biểu thức C++ có thể là hằng, hằng số hoặc biến, cũng có thể là giá trị trả về của hàm. Khác Pascal, một số toán tử C++ có tác dụng phụ, nghĩa là chúng sửa giá trị của toán hạng; các toán tử đó chỉ có thể tác động lên **giá trị trái**. Ban đầu, giá trị trái nghĩa là thứ có thể đứng bên trái phép gán; đặc điểm của nó là đại diện cho một vùng nhớ có thể sửa đổi.

Toán tử. Bảng sau đối chiếu một số toán tử thường dùng.

Tên toán tử	C++	Pascal
Cộng	+	+
Trừ	-	-
Nhân	*	*
Chia nguyên	/	div
Chia thực	/	/
Lấy dư	%	mod
Nhỏ hơn	<	<
Nhỏ hơn hoặc bằng	<=	<=
Lớn hơn	>	>
Lớn hơn hoặc bằng	>=	>=
Bằng	==	=
Khác	!=	<>
Phủ định bit	~	not
Phủ định logic	!	not
Và bit	&	and
Và logic	&&	and
Hoặc bit		or
Hoặc logic		or
Xor bit	^	xor
Dịch trái bit	<<	shl
Dịch phải bit	>>	shr

Có thể thấy một đặc điểm lớn của C++ là hầu như mọi toán tử đều được tạo bởi ký hiệu. Cách dùng các toán tử trong bảng gần giống Pascal, nhưng có ba điểm cần chú ý.

Thứ nhất, trong C++, chia nguyên và chia thực đều dùng /. Khi cả hai toán hạng là số nguyên thì thực hiện chia nguyên; khi có ít nhất một toán hạng là số thực thì thực hiện chia thực.

Thứ hai, kiểm tra bằng nhau phải viết hai dấu bằng, vì một dấu bằng được dùng làm phép gán. Người dùng Pascal ban đầu thường không quen điều này. Phức hơn nữa, phép gán trong C++ cũng trả về một giá trị. Nếu trong điều kiện nhằm kiểm tra bằng nhau thành một dấu bằng thì cú pháp vẫn hợp lệ nhưng ý nghĩa sai; trình biên dịch tốt thường cảnh báo phép gán trong điều kiện.

Thứ ba, toán tử bit và toán tử logic trong C++ là khác nhau.

C++ còn có một số toán tử mà Pascal không có. Với toán tử hai ngôi @, biểu thức gán $x = x @ y$ có thể viết gọn thành $x @= y$. Ví dụ $x += 39$ tương đương $x = x + 39$. Các toán tử hỗ trợ kiểu viết này gồm +, -, *, /, %, <<, >>, &, | và ^.

C++ cũng có ++ và --, đều là toán tử một ngôi, dùng để tăng hoặc giảm toán hạng một đơn vị. Chúng có dạng tiền tố và hậu tố. Dạng tiền tố tăng hoặc giảm biến trước rồi trả về giá trị biến; dạng hậu tố trả về giá trị biến trước rồi mới tăng hoặc giảm. Ví dụ nếu biến nguyên x có giá trị 12, biểu thức ++x có giá trị 13, còn x++ có giá trị 12, nhưng sau đó x cũng trở thành 13.

Một toán tử ba ngôi thú vị là ?: . Với biểu thức $\text{expr1} ? \text{expr2} : \text{expr3}$, C++ trước hết xét đúng sai của expr1; nếu đúng thì trả về expr2, nếu sai thì trả về expr3.

Chuyển kiểu cũng là một phép toán trong C++; cả $T(x)$ và $(T)x$ đều chuyển x sang giá trị kiểu T.

Các toán tử +, -, ++, --, +=, -= có thể dùng với con trỏ. ++ làm con trỏ trở tới vị trí kế tiếp ngay sau ô nhớ hiện tại; xét theo địa chỉ, lượng tăng phụ thuộc vào kích thước của kiểu mà con trỏ trỏ tới. Các phép toán khác suy ra tương tự. Dấu - còn có thể dùng giữa hai con trỏ cùng kiểu để trả về khoảng cách giữa chúng. Thao tác chỉ số mảng thực chất là một phép toán con trỏ: $p[i]$ tương đương $*(p + i)$, nên chỉ số cũng là một phép toán trong C++. Phép toán con trỏ thể hiện tính linh hoạt của C++, nhưng phép toán con trỏ sai có thể khiến chương trình truy cập địa chỉ bị cấm và sụp đổ.

Về độ ưu tiên toán tử, chỉ cần nhớ một số trường hợp thường gặp, chẳng hạn nhân chia cao hơn cộng trừ. Khác Pascal, trong C++ phép toán logic có độ ưu tiên thấp hơn phép toán quan hệ, nên một số ngoặc trong điều kiện phức hợp có thể bỏ qua. Khi không chắc, có thể thêm ngoặc để điều khiển thứ tự tính; bảng độ ưu tiên đầy đủ nên xem tài liệu tra cứu. Qua hệ thống toán tử có thể thấy C++ ngắn gọn và linh hoạt hơn Pascal.

Câu lệnh

Câu lệnh trong C++ hơi khác Pascal. Một câu lệnh có thể là định nghĩa, một biểu thức đơn, câu lệnh ghép, hoặc câu lệnh điều kiện và lặp sẽ nói dưới đây. Câu lệnh ghép trong Pascal được bao bởi begin và end; trong C++ chỉ cần thay bằng ngoặc nhọn trái và phải.

Câu lệnh điều kiện. Câu lệnh điều kiện trong C++ cũng là if, có hai dạng: có else và không có else. Khác Pascal, điều kiện cần đặt trong ngoặc, và sau đó không có then.

```
if(dieu_kien)
    cau_lenh
```

hoặc

```
if(dieu_kien)
    cau_lenh
else
    cau_lenh
```

C++ còn có câu lệnh switch tương ứng với case trong Pascal.

```

switch(bieu_thuc) {
    case hang:
        cau_lenh
        break;
    case hang:
        cau_lenh
        break;
    ...
    default:
        cau_lenh
}

```

Sau mỗi case có thể có nhiều câu lệnh đơn, và break cuối cùng không bắt buộc. Nếu sau một case không có break, sau khi thực hiện các câu lệnh của case đó chương trình sẽ tiếp tục thực hiện case ngay sau, cho đến khi gặp break hoặc hết switch. Điều này tiện khi muốn xử lý nhiều giá trị giống nhau.

```

switch(c) {
    case '0': case '1': case '2': case '3': case '4':
    case '5': case '6': case '7': case '8': case '9':
        // xu ly chu so
        break;
    ...
}

```

Nếu muốn mỗi case kết thúc là thoát khỏi switch, phải thêm break.

Câu lệnh lặp. Giống Pascal, C++ có ba loại vòng lặp, nhưng cú pháp khác nhiều.

```

while(dieu_kien)
    cau_lenh

do
    cau_lenh
while(dieu_kien)

```

Hai vòng lặp này đều tiếp tục chạy khi điều kiện đúng và dừng khi điều kiện sai. Khác biệt là vòng thứ nhất kiểm tra trước mỗi lượt lặp, còn vòng thứ hai kiểm tra sau mỗi lượt lặp.

Vòng for là một trong những câu lệnh khác biệt nhất giữa C++ và Pascal, đồng thời thể hiện rõ tính linh hoạt của C++.

```

for(khoi_tao; dieu_kien; thay_doi)
    cau_lenh

```

Các phần trong ngoặc được ngăn cách bằng dấu chấm phẩy. Trước khi vòng lặp bắt đầu, câu lệnh for thực hiện phần khởi tạo, thường dùng để khởi tạo biến lặp; biến được định nghĩa trong phần khởi tạo có vòng đời đến hết vòng lặp. Trước mỗi lượt lặp, for kiểm tra điều kiện; nếu đúng thì thực hiện thân vòng lặp, nếu sai thì thoát. Sau mỗi lượt lặp, phần thay đổi được thực hiện, thường dùng để sửa biến lặp.

```
int sum = 0;
for(int i = 1; i <= 100; i++)
    sum += i;
```

Tương tự Pascal, cả ba loại vòng lặp đều có thể dùng `break` và `continue` để thay đổi luồng điều khiển.

Khi nào dùng dấu chấm phẩy. Nói chung, mỗi câu lệnh trong C++ cần dấu chấm phẩy để kết thúc, kể cả câu lệnh nằm giữa `if` và `else`; điểm này khác Pascal. Nhưng nếu đó là câu lệnh ghép, sau ngoặc nhọn phải không cần dấu chấm phẩy. Điều này chỉ áp dụng cho ngoặc nhọn biểu diễn câu lệnh ghép; nếu là định nghĩa kiểu liệt kê, định nghĩa cấu trúc hoặc khởi tạo mảng, sau ngoặc nhọn ở vị trí vốn cần dấu chấm phẩy vẫn phải có dấu chấm phẩy.

1.5 Hàm

Trong C++, định nghĩa một hàm có dạng:

```
kieu_tra_ve ten_ham(danh_sach_tham_so) {
    cau_lenh
}
```

So với Pascal, C++ không có khái niệm thủ tục; thủ tục chính là hàm có kiểu trả về `void`, tức không trả về gì. Khác Pascal, hàm C++ không thể được định nghĩa lồng trong một hàm khác. Khi vào một hàm, biến có thể dùng chỉ gồm biến toàn cục và các biến chỉ ra trong danh sách tham số. Trong Pascal, nếu thủ tục hoặc hàm không có tham số thì bỏ ngoặc của danh sách tham số, còn trong C++ ngoặc luôn luôn phải có. Các mục trong danh sách tham số ngăn cách bằng dấu phẩy, hình thức giống định nghĩa biến.

```
int f(int a, int b) {
    ...
}
```

Tương tự thao tác chỉ số, lời gọi hàm trong C++ cũng được xem là một phép toán; các toán hạng là bản thân hàm và các tham số thực, kết quả là giá trị trả về của hàm.

Cách truyền tham số

Trong Pascal có tham trị và tham biến. Trong C++, tham số thông thường được truyền theo kiểu tham trị, còn tham số định nghĩa bằng tham chiếu được truyền theo kiểu tham biến. Vì truyền tham trị phải sao chép toàn bộ giá trị của biến, với biến lớn mà hàm bảo đảm không sửa giá trị, ta có thể truyền bằng tham chiếu hằng.

```
int f(int a, int& b, const int& c) {
    ...
}
```

Ở đây `a` được truyền theo tham trị, `b` được truyền theo tham biến, còn `c` cũng truyền địa chỉ biến nhưng nhờ `const` nên trong hàm `f` giá trị của `c` không thể bị thay đổi. Mọi câu lệnh trong `f` cố sửa `c` sẽ khiến trình biên dịch báo lỗi. Giống Pascal, khi gọi hàm, vị trí tham biến chỉ có thể thay bằng biến, hay

chính xác hơn là giá trị trái. Tuy nhiên, nếu tham số là tham chiếu hằng thì biểu thức thông thường cũng được; hệ thống sẽ tự tạo biến tạm cho nó.

Mảng trong C++ được truyền dưới dạng con trỏ. Vì vậy mọi thay đổi một phần tử mảng trong thân hàm sẽ làm thay đổi phần tử tương ứng của mảng lúc gọi, vì chúng có cùng địa chỉ trong bộ nhớ.

Trả về giá trị hàm

Khác Pascal, giá trị hàm trong C++ được trả bằng câu lệnh `return`. Câu lệnh này vừa trả về giá trị hàm, vừa kết thúc việc thực hiện hàm và quay lại vị trí gọi hàm. Dạng của nó là:

```
return gia_tri_tra_ve;
```

Hàm kiểu `void` có thể dùng trực tiếp `return`; để kết thúc.

Hàm nội tuyến

Thêm `inline` trước định nghĩa hàm thường sẽ tạo ra định nghĩa hàm nội tuyến. Khi đó trình biên dịch cố gắng chép trực tiếp các lệnh trong hàm vào nơi gọi, thay vì truyền tham số và nhảy lệnh. Vì vậy, các hàm đơn giản và được dùng nhiều lần thường được định nghĩa nội tuyến.

```
inline int max(int x, int y) {  
    if(x > y)  
        return x;  
    else  
        return y;  
}
```

Nếu hàm nội tuyến không thể được mở trực tiếp, chẳng hạn trong hàm có cấu trúc phức tạp như vòng lặp, hiệu quả sẽ giống hàm thường.

Nạp chồng hàm và tham số mặc định

C++ hỗ trợ nạp chồng hàm và tham số mặc định; cả hai đều thể hiện sự ngắn gọn và linh hoạt của ngôn ngữ.

Nạp chồng hàm nghĩa là các hàm cùng tên có thể được định nghĩa nhiều lần, nhưng mỗi lần có danh sách tham số khác nhau. Khi hàm được gọi, trình biên dịch dựa vào danh sách tham số thực để quyết định gọi hàm nào. Thông thường các hàm được nạp chồng dùng để làm cùng một việc, chỉ khác đôi chút về tham số.

```
void print(int i) {  
    cout << i << endl;  
}  
void print(const char* s) {  
    cout << s << endl;  
}  
int main() {  
    print(39);           // gọi hàm thu nhất  
    print("Hello, world!"); // gọi hàm thu hai  
}
```

Tham số mặc định là việc chỉ định trước giá trị mặc định cho một số tham số khi định nghĩa hàm. Khi gọi, nếu các tham số này có giá trị bằng mặc định thì có thể bỏ qua. Các tham số có mặc định chỉ được đặt ở cuối danh sách tham số.

```
void inc(int& x, int d = 1) {
    x += d;
}

int main() {
    int i = 1;
    inc(i);    // tương đương inc(i, 1);
    inc(i, 3);
}
```

Với cả nạp chồng hàm và tham số mặc định, cần tránh tạo ra sự mơ hồ. Vì hai tính năng này không quá quan trọng trong thi tin học, bài viết chỉ nhắc qua; chi tiết nên xem tài liệu khác.

1.6 Một số hàm thư viện thường dùng

Như đã nói trong lời nói đầu, C++ chuẩn cung cấp thư viện rất mạnh. Mục này chỉ giới thiệu vài hàm thư viện có chức năng gần với các thủ tục và hàm chuẩn của Pascal.

Định nghĩa hàm	Tập tin đầu	Tác dụng	Ghi chú
<code>void* memset(void* p, int b, size_t n);</code>	<code>cstring</code>	Đặt n byte liên tiếp bắt đầu tại p thành giá trị b	Giống FillChar, chú ý thứ tự tham số
<code>void* memmove(void* p, const void* q, size_t n);</code>	<code>cstring</code>	Sao chép n byte từ vùng q đến vùng p	Giống Move; hai vùng có thể chồng lấn một phần
<code>double atof(const char* p);</code> <code>int atoi(const char* p);</code> <code>long atol(const char* p);</code>	<code>cstdlib</code>	Chuyển chuỗi p thành số được biểu diễn	Giống Val
<code>double fabs(double);</code>	<code>cmath</code>	Hàm trị tuyệt đối	Giống Abs
<code>double ceil(double);</code>	<code>cmath</code>	Lấy nguyên, lần lượt là làm tròn lên và làm tròn xuống	
<code>double floor(double);</code>			
<code>double sqrt(double);</code>	<code>cmath</code>	Căn bậc hai	Giống Sqrt
<code>double pow(double d, double e);</code>	<code>cmath</code>	Lũy thừa, trả về d^e	
<code>double sin(double);</code>	<code>cmath</code>	Hàm lượng giác	
<code>double cos(double);</code>			
<code>double tan(double);</code>			
<code>double asin(double);</code>	<code>cmath</code>	Hàm lượng giác ngược	
<code>double acos(double);</code>			
<code>double atan(double);</code>			
<code>double atan2(double y, double x);</code>	<code>cmath</code>	Hàm arctan tăng cường, trả về góc cực của điểm (x, y)	Rất hữu ích, tự xét góc phần tư
<code>double sinh(double);</code>	<code>cmath</code>	Hàm hyperbol	
<code>double cosh(double);</code>			
<code>double tanh(double);</code>			
<code>double exp(double);</code>	<code>cmath</code>	Hàm mũ cơ số e	Giống Exp
<code>double log(double);</code>	<code>cmath</code>	Logarit cơ số e và cơ số 10	Log giống Ln
<code>double log10(double);</code>			

C++ chuẩn không cung cấp hàm Pi. Muốn lấy giá trị π , thường viết:

```
const double pi = acos(0.) * 2;
```

Chi tiết hơn về các hàm thư viện nên xem tài liệu liên quan.

1.7 Kết chương

Sau khi nắm các mục trên, về cơ bản ta đã có thể dùng C++ để làm những việc thường làm bằng Pascal. Mục này dùng C++ để giải một bài thực tế: bài 3 bảng nâng cao NOIP 2003, *Cây nhị phân cộng điểm*.

Mô tả bài toán. Cho một cây nhị phân T có n đỉnh, duyệt giữa của nó là $(1, 2, 3, \dots, n)$, trong đó các số là mã đỉnh. Mỗi đỉnh có một điểm là số nguyên dương; điểm của đỉnh thứ j ký hiệu d_j . Cây T và mọi cây con của nó đều có một giá trị cộng điểm. Với cây con S , giá trị này bằng giá trị cộng điểm của cây con trái nhân với giá trị cộng điểm của cây con phải, rồi cộng điểm của gốc S .

Nếu cây con rỗng thì giá trị cộng điểm quy ước là 1. Với lá, giá trị cộng điểm chính là điểm của lá, không xét hai cây con rỗng. Hãy tìm một cây nhị phân có duyệt giữa $(1, 2, 3, \dots, n)$ và giá trị cộng điểm lớn nhất; cần in giá trị lớn nhất và duyệt trước của cây đó.

Dữ liệu vào. Dòng thứ nhất là số nguyên $n < 30$, số đỉnh. Dòng thứ hai gồm n số nguyên cách nhau bởi khoảng trắng, là điểm của từng đỉnh, mỗi điểm nhỏ hơn 100.

Dữ liệu ra. Dòng thứ nhất là giá trị cộng điểm lớn nhất, kết quả không vượt quá 4 000 000 000. Dòng thứ hai gồm n số nguyên cách nhau bởi khoảng trắng, là duyệt trước của cây.

Input

5

5 7 1 2 10

Output

145

3 1 2 4 5

Để đơn giản, chương trình dưới đây dùng bàn phím và màn hình. Nhập chuẩn trong C++ cũng rất đơn giản: cin biểu diễn đầu vào chuẩn, thường là bàn phím, và thao tác trích bằng >> thực hiện nhập.

Từ mô tả bài toán có thể thấy khi tìm giá trị cộng điểm lớn nhất của T , có nhiều bài toán con là tìm giá trị của các cây con, và các bài toán con này cũng thỏa mãn tính tối ưu. Trong duyệt giữa, các đỉnh của cùng một cây con có chỉ số liên tiếp, nên chỉ cần chỉ ra vị trí đầu và cuối là mô tả đầy đủ một bài toán con. Vì vậy có thể dùng quy hoạch động. Gọi $best_{i,j}$ là giá trị cộng điểm lớn nhất của cây con từ đỉnh i đến đỉnh j . Khi đó:

$$best_{i,j} = \begin{cases} d_j, & 1 \leq i = j \leq n, \\ \max \{ best_{i+1,j} + d_i, best_{i,j-1} + d_j, \max_{i < k < j} (best_{i,k-1} best_{k+1,j} + d_k) \}, & 1 \leq i < j \leq n. \end{cases}$$

Vì đề còn yêu cầu in duyệt trước của cây có điểm lớn nhất, ta dùng $root_{i,j}$ để ghi chỉ số gốc của cây con từ i đến j . Cuối cùng dùng đệ quy để in duyệt trước.

```
//: C01:Tree.cpp
```

```
// Binary Tree with Scores (NOIP2003 Advanced Division)
```

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
const int max_nodes = 30;
```

```
int n;
```



```

unsigned int best[max_nodes][max_nodes];
int root[max_nodes][max_nodes];
bool first;
void visit(int i, int j) {
    if(i > j)
        return;
    if(first)
        first = false;
    else
        cout << ' ';
    cout << root[i][j] + 1;
    visit(i, root[i][j] - 1);
    visit(root[i][j] + 1, j);
}
int main() {
    cin >> n;
    memset(best, 0, sizeof(best));
    memset(root, 0xFF, sizeof(root));
    for(int i = 0; i < n; i++) {
        cin >> best[i][i];
        root[i][i] = i;
    }
    for(int d = 1; d < n; d++)
        for(int i = 0; i + d < n; i++) {
            int j = i + d;
            if(best[i + 1][j] > best[i][j - 1]) {
                best[i][j] = best[i][i] + best[i + 1][j];
                root[i][j] = i;
            }
            else {
                best[i][j] = best[j][j] + best[i][j - 1];
                root[i][j] = j;
            }
            for(int k = i + 1; k < j; k++) {
                unsigned int value = best[k][k]
                    + best[i][k - 1] * best[k + 1][j];
                if(value > best[i][j]) {
                    best[i][j] = value;
                    root[i][j] = k;
                }
            }
        }
    cout << best[0][n - 1] << endl;
    first = true;
    visit(0, n - 1);
    cout << endl;
}
///  


```

Cần chú ý rằng chỉ số mảng trong C++ bắt đầu từ 0, nên chương trình đã điều chỉnh chỉ số tương ứng. Qua chương này có thể thấy C++ và Pascal có nhiều điểm giống nhau, nhưng cũng có nhiều khác biệt. Ta nên dùng phương pháp loại suy: lấy điểm giống làm nền tảng, rồi nắm chắc các điểm khác biệt; thông qua luyện tập liên tục, cuối cùng sẽ sử dụng C++ thành thạo.

2 Đi sâu hơn vào ngôn ngữ C++

Điều kiện cần để đọc chương này là biết viết chương trình C++ đơn giản và có một lượng kiến thức thuật toán nhất định. Chương này giới thiệu lớp và đối tượng trong C++, cùng một số lớp thường dùng của thư viện chuẩn.

2.1 Lớp

Lớp không được dùng trực tiếp quá nhiều trong thi tin học. Tuy nhiên, các thư viện mạnh của C++ đều được xây dựng trên cơ sở lớp, nên ta cần có cái nhìn cơ bản về chúng. Trong mục này, hai thuật ngữ **lớp** và **đối tượng** xuất hiện thường xuyên. Lớp thực chất là một kiểu, còn đối tượng là một biến có kiểu là một lớp nào đó.

Định nghĩa lớp

Nói đơn giản, lớp là sự mở rộng của kiểu cấu trúc. Thêm một số hàm vào cấu trúc là có lớp đơn giản nhất.

```
struct Date {  
    int d, m, y;  
    void init(int dd, int mm, int yy);  
    void add_year(int n);  
    void add_month(int n);  
    void add_day(int n);  
};
```

Khi đó ta có thể truy cập hàm thành viên giống như truy cập biến thành viên.

```
Date national_day;  
national_day.init(1, 10, 2003);  
Date next_day;  
next_day = national_day;  
next_day.add_day(1);
```

Khi định nghĩa hàm thành viên của lớp ở bên ngoài thân lớp, để tránh trùng tên hàm giữa các lớp khác nhau, phải thêm tên lớp trước tên hàm và ngăn cách bằng ::.

```
void Date::init(int dd, int mm, int yy) {  
    d = dd;  
    m = mm;  
    y = yy;  
}
```

Trong hàm thành viên của lớp, muốn dùng biến thành viên chỉ cần viết trực tiếp tên biến.

Quyền truy cập

Một tư tưởng quan trọng của lập trình hướng đối tượng là đóng gói dữ liệu, tức giấu dữ liệu bên trong đối tượng và chỉ thao tác lên đối tượng thông qua hàm thành viên. Khi đó cần chỉ định quyền truy cập cho các thành viên trong lớp.

```
struct Date {
private:
    int d, m, y;
public:
    void init(int dd, int mm, int yy);
    void add_year(int n);
    void add_month(int n);
    void add_day(int n);
};
```

Ở đây, các biến thành viên `d`, `m`, `y` có quyền riêng tư, chỉ hàm thành viên của chính lớp mới truy cập được. Bốn hàm thành viên phía dưới có quyền công khai, bên ngoài đều có thể gọi. Dĩ nhiên không phải biến thành viên lúc nào cũng phải riêng tư, hay hàm thành viên lúc nào cũng phải công khai; một số hàm thành viên dùng nội bộ cũng có thể là riêng tư, và biến thành viên cũng có thể công khai.

Khi định nghĩa lớp, ta thường dùng `class` thay cho `struct`.

```
class Date {
private:
    int d, m, y;
public:
    void init(int dd, int mm, int yy);
    void add_year(int n);
    void add_month(int n);
    void add_day(int n);
};
```

Khác biệt duy nhất giữa `class` và `struct` là khi không chỉ định quyền truy cập, `struct` mặc định công khai còn `class` mặc định riêng tư.

Hàm dựng và hàm hủy

Trong một lớp có thể định nghĩa hai loại hàm đặc biệt. Loại thứ nhất là hàm dựng: không có kiểu trả về, cùng tên với lớp, được gọi tự động khi đối tượng được tạo, thường dùng để khởi tạo đối tượng; sau danh sách tham số của nó có thể có danh sách khởi tạo. Loại thứ hai là hàm hủy: cũng không có kiểu trả về, tên gồm dấu `~` đặt trước tên lớp, được gọi tự động khi đối tượng kết thúc vòng đời, thường dùng để dọn dẹp. Hàm dựng có thể có tham số và có thể được nạp chồng; hàm hủy không có tham số.

```
class Date {
public:
    Date(int dd, int mm, int yy);
    Date(char* date);
    ~Date();
};
```

```

Date::Date(int dd, int mm, int yy) : d(dd), m(mm), y(yy) {
}
Date national_day(1, 10, 2003);
Date christmas("December 25th, 2003");

```

Trong hiện thực hàm dựng của lớp Date, phần sau dấu hai chấm là danh sách khởi tạo, có thể dùng để khởi tạo biến thành viên. Cũng có thể khởi tạo trong thân hàm dựng, nhưng thông thường nên dùng danh sách khởi tạo.

Thành viên tĩnh

Trong một lớp, có những biến thành viên được chia sẻ bởi mọi đối tượng của lớp. Có thể dùng biến toàn cục, nhưng làm như vậy phá vỡ đóng gói. Để giải quyết, ta định nghĩa chúng là biến thành viên tĩnh. Cũng có những hàm thành viên có thể được gọi mà không gắn với một đối tượng cụ thể; ta định nghĩa chúng là hàm thành viên tĩnh. Chỉ cần thêm static trước định nghĩa thành viên thông thường.

Thành viên tĩnh vẫn tuân theo quyền truy cập của lớp. Biến thành viên tĩnh phải được khởi tạo bên ngoài định nghĩa lớp. Hàm thành viên tĩnh không được dùng biến thành viên thường, cũng không được gọi hàm thành viên thường, nhưng có thể dùng biến thành viên tĩnh và gọi hàm thành viên tĩnh khác. Hàm thành viên thường có thể dùng biến thành viên tĩnh và gọi hàm thành viên tĩnh. Ví dụ sau minh họa cách định nghĩa và dùng thành viên tĩnh.

```

//: C02:Counter.cpp
// Static members in class
#include <iostream>
using namespace std;
class Counter {
public:
    Counter(int id2);
    ~Counter();
    int get_id();
    static int get_tot();
private:
    int id;
    static int tot;
};
int Counter::tot = 0;
Counter::Counter(int id2) :id(id2) {
    tot++;
}
Counter::~~Counter() {
    tot--;
}
int Counter::get_id() {
    return id;
}
int Counter::get_tot() {
    return tot;
}

```

```

int main() {
    Counter c1(0), c2(1);
    cout << c1.get_id() << endl;
    cout << c1.get_tot() << endl;
    Counter c3(2);
    cout << Counter::get_tot() << endl;
} ///:~

```

Hàm thành viên hằng

Trong một lớp, có những hàm thành viên không thay đổi trạng thái đối tượng, chẳng hạn `get_id()` của lớp `Counter` ở trên. Khi đó có thể thêm `const` sau danh sách tham số của định nghĩa hàm thành viên để khai báo chúng là hàm thành viên hằng. Trong hàm thành viên hằng, không được viết câu lệnh sửa biến thành viên; nếu có, trình biên dịch sẽ báo lỗi.

Ngoài ra, đối tượng hằng chỉ có thể dùng hàm thành viên hằng. Ví dụ ở chương trước, nếu truyền đối tượng bằng tham chiếu hằng, trong hàm chỉ có thể dùng các hàm thành viên hằng của đối tượng đó. Hàm thành viên tĩnh không thể được định nghĩa là hàm thành viên hằng. Sửa chương trình trên như sau:

```

//: C02:Counter2.cpp
// Constant member functions in class
#include <iostream>
using namespace std;
class Counter {
public:
    Counter(int id2) : id(id2) {
        tot++;
    }
    ~Counter() {
        tot--;
    }
    int get_id() const {
        return id;
    };
    static int get_tot() {
        return tot;
    }
private:
    int id;
    static int tot;
};
int Counter::tot = 0;
int main() {
    const Counter c1(0), c2(1);
    cout << c1.get_id() << endl;
    cout << c1.get_tot() << endl;
    Counter c3(2);
    cout << Counter::get_tot() << endl;
} ///:~

```

Từ chương trình này cũng thấy rằng hiện thực hàm thành viên có thể đặt ngay trong định nghĩa lớp. Nếu làm vậy, trình biên dịch sẽ cố xử lý chúng theo cách nội tuyến. Thông thường các hàm thành viên ngăn được hiện thực đầy đủ trong định nghĩa lớp, còn hàm dài thì hiện thực bên ngoài.

Con trỏ this

Đôi khi trong hàm thành viên của một đối tượng ta cần tham chiếu đến chính đối tượng đó; khi ấy dùng con trỏ `this`. Trong hàm thành viên, `this` là con trỏ tới kiểu của đối tượng và luôn trỏ tới đối tượng hiện tại. Ngoài ra, nếu trong hàm thành viên có biến cục bộ trùng tên với biến thành viên, viết trực tiếp tên biến sẽ chỉ biến cục bộ. Muốn truy cập biến thành viên thì dùng `this`, chẳng hạn `this->x`.

Trên đây là phần giới thiệu ngắn về lớp trong C++. Thực ra lợi ích lớn nhất của lớp và đối tượng là đơn giản hóa đáng kể việc viết và bảo trì phần mềm lớn. Trong tin học, dùng lớp và đối tượng đúng mức cũng có thể giảm độ phức tạp khi lập trình.

2.2 Nạp chồng toán tử

Nạp chồng toán tử nghĩa là định nghĩa lại các toán tử mà ngôn ngữ cung cấp. Mục đích ban đầu của kỹ thuật này chỉ là làm mã ngắn gọn hơn. Chẳng hạn ta viết một lớp biểu diễn số nguyên độ chính xác cao, và đối tượng của lớp này cần hỗ trợ nhiều phép toán số học. Nếu mọi phép toán đều viết thành hàm, mã sẽ kém đẹp: biểu thức đơn giản $x + y * z$ có thể phải viết thành `add(x, multiply(y, z))`. Nếu có thể định nghĩa lại toán tử của ngôn ngữ, ta dùng kiểu tự định nghĩa giống như dùng kiểu dựng sẵn.

Như chương trước đã nói, toán tử trong C++ rất phong phú, đến cả chỉ số và lời gọi hàm cũng có thể xem là toán tử. Khi nạp chồng toán tử, ngoài toán tử số học thông thường, hầu như mọi toán tử đều có thể được định nghĩa lại.

Dĩ nhiên nạp chồng toán tử có một số ràng buộc. Trước hết, chỉ có thể nạp chồng toán tử đã tồn tại trong C++, không thể tự định nghĩa toán tử mới. Thứ hai, toán tử được nạp chồng phải có ít nhất một toán hạng thuộc kiểu do người dùng định nghĩa; tức không thể nạp chồng toán tử mà mọi toán hạng đều là kiểu dựng sẵn. Thứ ba, toán tử sau khi nạp chồng vẫn giữ độ ưu tiên ban đầu. Ngoài ra, một số toán tử như `.` và `?:` không thể nạp chồng.

Nói chung, chức năng của toán tử được nạp chồng nên phù hợp với ý nghĩa ban đầu của nó. Điều này không tuyệt đối: trong nhập xuất bằng luồng, toán tử trích `>>` và toán tử chèn `<<` đã thay đổi hoàn toàn chức năng ban đầu, nhưng sự thay đổi ấy rất khéo.

Có thể xem toán tử là một loại hàm đặc biệt, còn toán hạng là tham số của hàm. Vì vậy ta có thể nạp chồng toán tử giống như định nghĩa hàm. Tên hàm của toán tử là `operator` cộng với chính toán tử đó. Với toán tử hai ngôi `@`, biểu thức `a @ b` tương đương `operator @(a, b)` hoặc `a.operator @(b)`. Với toán tử một ngôi `@`, biểu thức `@a` tương đương `operator @(a)` hoặc `a.operator @()`. Nói cách khác, toán tử được nạp chồng có thể là hàm toàn cục hoặc hàm thành viên của lớp.

```
class complex {
    double re, im;
public:
    complex(double r, double i) : re(r), im(i) {}
    double real() const { return re; }
    double imag() const { return im; }
    complex operator +(const complex& x) {
        return complex(re + x.re, im + x.im);
    }
}
```

```
};
complex operator -(const complex& x, const complex& y) {
    return complex(x.real() - y.real(), x.imag() - y.imag());
}
```

Trong đoạn trên, toán tử cộng được định nghĩa là hàm thành viên, còn toán tử trừ được định nghĩa là hàm toàn cục. Thông thường toán tử hai ngôi phổ thông được định nghĩa thành hàm toàn cục, toán tử một ngôi được định nghĩa thành hàm thành viên. Tuy nhiên các toán tử trả về kết quả là giá trị trái, như toán tử gán, thường cũng được định nghĩa thành hàm thành viên và kiểu trả về phải là tham chiếu.

Một số toán tử đặc biệt

Toán tử chỉ số. Toán tử chỉ số `[]` bắt buộc phải được định nghĩa là hàm thành viên của lớp, thường được nạp chồng khi định nghĩa lớp giống mảng. Điều thú vị là sau khi nạp chồng, giá trị trong ngoặc vuông không nhất thiết là số nguyên; điều này rất thuận tiện cho lớp ánh xạ trong STL.

Toán tử lời gọi hàm. Toán tử lời gọi hàm `()` bắt buộc phải được định nghĩa là hàm thành viên của lớp; số lượng và kiểu tham số không bị hạn chế. Nói chung, nó thường được dùng để định nghĩa mảng nhiều chiều hoặc các đối tượng có thể sử dụng như hàm.

Toán tử tăng giảm. Toán tử `++` và `--` có hai cách dùng: tiền tố và hậu tố. Nạp chồng dạng tiền tố giống toán tử một ngôi thông thường. Dạng hậu tố được thêm một tham số nguyên giả, nhờ đó có thể phân biệt với dạng tiền tố theo quy tắc nạp chồng hàm.

```
class counter {
    int v;
public:
    counter(int val = 0) : v(val) { }
    int get_value() const { return v; }

    counter& operator ++() { // dạng tiền tố
        v++;
        return *this;
    }
    counter operator ++(int) { // dạng hậu tố
        counter temp = *this;
        v++;
        return temp;
    }
};
```

Các ví dụ cụ thể về nạp chồng toán tử sẽ còn xuất hiện ở những phần sau.

2.3 Chuỗi

Thư viện chuẩn C++ cung cấp lớp `string` để xử lý chuỗi thông thường. Lớp này được định nghĩa trong tập tin đầu `string`, vì vậy khi dùng đừng quên đưa tập tin đầu này vào.

Thực ra để hỗ trợ các bộ ký tự khác, C++ trước hết dùng khuôn mẫu để hiện thực một lớp chuỗi tổng quát `basic_string`; `string` chỉ là một bản đặc biệt của nó với kiểu ký tự là `char`. Để tiện, mục này xem `string` như một lớp độc lập và giới thiệu một số hàm thường dùng.

Trước hết là vài điểm cơ bản. Chuỗi của lớp `string` về lý thuyết không bị giới hạn độ dài; nó tự động cấp phát không gian theo độ dài thực tế. Kiểu `string::size_type` thực chất là `unsigned int`, dùng để mô tả vị trí và độ dài trong chuỗi. `string::npos` là hằng thành viên tĩnh có kiểu `size_type`, giá trị là giá trị lớn nhất của `unsigned int`, thường dùng để mô tả độ dài lớn nhất. Với đối tượng `string s`, ta dùng `s[i]` để truy cập ký tự thứ *i*; giống chỉ số mảng trong C++, ký tự cũng được đánh số từ 0.

Nhờ nạp chồng toán tử, phép gán, nối và so sánh của `string` không khác mấy kiểu `string` trong Pascal. Đáng chú ý là nơi cần một chuỗi khác không chỉ nhận đối tượng `string`, mà cả con trỏ ký tự kế thừa từ C cũng có thể dùng. Ngoài ra, hàm thành viên `c_str()` trả về một con trỏ ký tự có nội dung giống đối tượng `string`. Các thủ tục hoặc hàm chuỗi của Pascal đều có hàm thành viên tương ứng trong `string`.

Pascal	Tác dụng	<code>string</code>
Length	Lấy độ dài chuỗi	<code>length</code>
Copy	Lấy chuỗi con	<code>substr</code>
Insert	Chèn một đoạn chuỗi	<code>insert</code>
Delete	Xóa một đoạn chuỗi	<code>erase</code>
Pos	Tìm vị trí xuất hiện của chuỗi con	<code>find</code>

Các hàm thành viên này đều có nhiều phiên bản nạp chồng để dùng linh hoạt. Dưới đây là các cách dùng thường gặp nhất:

```
size_type length() const;
string substr(size_type pos = 0, size_type n = npos) const;
string& insert(size_type pos, const string& s);
string& erase(size_type pos = 0, size_type n = npos);
size_type find(const string& s) const;
```

Trong danh sách tham số trên, `pos` là vị trí trong chuỗi gốc, `n` là độ dài chuỗi con, còn `s` là một chuỗi khác. Nếu `n` lớn hơn độ dài còn lại của chuỗi, chuỗi con sẽ đi từ `pos` đến hết chuỗi gốc. Nếu không chỉ định `n`, tham số mặc định `npos` sẽ lấy chuỗi con đến hết chuỗi gốc.

Khác Pascal, vì 0 là vị trí hợp lệ trong chuỗi `string`, khi tìm chuỗi con mà không thấy, `find` trả về `npos` chứ không trả về 0. Hàm `rfind` cũng tìm chuỗi con, nhưng khác `find` ở chỗ nó trả về vị trí xuất hiện cuối cùng, còn `find` trả về vị trí xuất hiện đầu tiên. Trên đây chỉ là các hàm thành viên thường dùng nhất của `string`; chi tiết cần xem tài liệu.

2.4 Luồng

Trong C++, nhập xuất được hiện thực bằng luồng. Thiết kế các lớp luồng của thư viện chuẩn C++ thể hiện rõ tính linh hoạt của ngôn ngữ và ưu thế của lập trình hướng đối tượng.

Trong C++, lớp cơ sở của mọi luồng nhập là `istream`, lớp cơ sở của mọi luồng xuất là `ostream`. Luồng nhập chuẩn `cin`, luồng nhập tập tin và luồng nhập chuỗi đều dẫn xuất từ `istream`, nên nơi nào dùng được `istream` thì các luồng nhập này đều dùng được; luồng xuất cũng tương tự.

Thao tác thường dùng nhất với luồng là nhập và xuất. Trong các ví dụ trước, với một luồng nhập ta dùng toán tử trích `>>` để nhập, còn với một luồng xuất ta dùng toán tử chèn `<<` để xuất. Các thao tác này gần như dùng được với mọi kiểu dựng sẵn của ngôn ngữ. Cần chú ý rằng chúng tự động xử lý định dạng

dữ liệu theo kiểu. Đặc biệt khi nhập ký tự và chuỗi, gồm cả con trỏ ký tự và lớp `string`, toán tử trích trước hết bỏ qua mọi ký tự trắng như khoảng trắng, tab và xuống dòng, rồi mới trích dữ liệu. Với chuỗi, khi gặp lại ký tự trắng thì thao tác trích kết thúc.

Với một số yêu cầu nhập đặc biệt, ta có thể dùng nhập không định dạng. Các hàm thành viên thường dùng liên quan đến nhập không định dạng gồm:

```
int get();
istream& get(int& c);
istream& get(char* p, int n, char term = '\n');
istream& getline(char* p, int n, char term = '\n');
```

Hai hàm đầu đều đọc một ký tự từ luồng, kể cả ký tự trắng. Chú ý chỗ lẽ ra là `char` lại dùng `int`, chủ yếu vì khi đọc đến cuối luồng chúng phải trả về `-1`. Hai hàm sau đọc nhiều ký tự từ luồng, tối đa n ký tự, nhưng nếu gặp `term` thì kết thúc sớm; cả hai đòi hỏi vùng `p` có đủ không gian. Khác biệt là `get()` vẫn để ký tự `term` trong luồng, còn `getline()` bỏ luôn ký tự đó, điều này ảnh hưởng đến các lần nhập sau.

Hàm thành viên `peek()` trả về ký tự kế tiếp sẽ được đọc từ luồng nhưng không thật sự lấy nó ra, tức ký tự đó vẫn nằm trong luồng. Hàm `unget()` đẩy ký tự vừa đọc gần nhất trở lại luồng. Hàm `eof()` kiểm tra luồng đã đến cuối chưa, giống `Eof` của Pascal. Có lúc cũng có thể dùng trực tiếp đối tượng luồng. Ở nơi cần giá trị logic, chẳng hạn trong ngoặc của `if`, C++ tự chuyển luồng thành giá trị logic biểu thị luồng còn bình thường không. Nếu đã đọc đến cuối luồng, giá trị logic này thành sai. Ví dụ, đoạn sau liên tục đọc số nguyên đến cuối luồng:

```
int i;
while(cin >> i) {
    // Do something
}
```

Các hàm thành viên của luồng không có hàm giống `Eoln` trong Pascal, nhưng có thể kiểm tra cuối dòng bằng `cin.peek() == '\n'`.

Xuất định dạng

Trong Pascal, khi xuất ta có thể chỉ định độ rộng trường và số chữ số phần thập phân. Luồng của C++ cũng làm được. Chỉ định độ rộng trường dùng hàm thành viên `width()`. Chỉ định số chữ số phần thập phân hơi phiền hơn: trước hết đặt cách xuất số thực về dạng cố định, sau đó đặt số chữ số phần thập phân.

```
cout.setf(ios::fixed, ios::floatfield);
cout.precision(2);
cout << 1.2345 << endl;
```

Câu lệnh đầu đặt cách xuất số thực thành dạng cố định, câu lệnh thứ hai đặt số chữ số phần thập phân là 2. Giống Pascal, chữ số cuối của phần thập phân được làm tròn.

Cần chú ý `width()` chỉ có hiệu lực với lần xuất định dạng kế tiếp. Nếu nhiều lần xuất cần chỉ định độ rộng trường thì phải gọi `width()` nhiều lần. Còn `precision()` có hiệu lực với mọi lần xuất số thực sau đó.

```
cout.width(3);
cout << 1 << 2; // 1 có độ rộng 3, 2 dung độ rộng thực
cout.width(4);
cout << 2;
```

Nếu thấy cách viết này phiền, có thể viết:

```
cout << setw(3) << 1 << 2 << setw(4) << 2;
```

Khi đó đừng quên thêm `#include <iomanip>` ở đầu chương trình, vì `setw` được định nghĩa trong `iomanip`. Các nội dung xuất định dạng của luồng C++ còn nhiều, có thể xem thêm tài liệu.

Luồng tập tin và luồng chuỗi

Một ưu điểm lớn của việc C++ trừu tượng hóa nhập xuất thành luồng là ta không cần quan tâm luồng đang thao tác tồn tại dưới hình thức nào. Khi nhập, nguồn của luồng là bàn phím, tập tin hay nội dung trong một chuỗi thì ngoài cách tạo luồng ban đầu có chút khác biệt, các thao tác sau đó đều giống nhau.

Luồng tập tin. Trong C++, lớp của luồng nhập tập tin là `ifstream`, lớp của luồng xuất tập tin là `ofstream`, cả hai được định nghĩa trong tập tin đầu `fstream`.

Để mở một tập tin, có thể chỉ định tên tập tin khi tạo luồng, hoặc tạo xong rồi dùng hàm thành viên `open()`.

```
ifstream fin("test.in"); // chỉ định khi tạo, dung duoc ngay
ofstream fout;
fout.open("test.out"); // chỉ định sau khi tạo
```

Sau khi luồng tập tin được mở, ta dùng chúng giống như `cin` hoặc `cout`. Thông thường không cần viết riêng câu lệnh đóng luồng tập tin, vì hàm hủy của lớp luồng tập tin sẽ tự động đóng tập tin. Nếu muốn đóng trước khi hàm hủy chạy, có thể dùng hàm thành viên `close()`.

Luồng chuỗi. Luồng chuỗi trong C++ dùng để đọc dữ liệu từ một chuỗi, hoặc xuất dữ liệu vào một chuỗi. Lớp của luồng nhập chuỗi là `istream`, lớp của luồng xuất chuỗi là `ostream`, cả hai được định nghĩa trong tập tin đầu `sstream`.

Khi tạo luồng nhập chuỗi, ta cần chỉ định một chuỗi làm nguồn dữ liệu. Trong quá trình sử dụng, có thể gọi hàm thành viên `str()` bất cứ lúc nào để lấy chuỗi tương ứng. Luồng chuỗi có thể dùng để chuyển đổi giữa dữ liệu kiểu cụ thể và chuỗi.

```
string s("123");
istream sin(s);
int i;
sin >> i;
cout << i << endl;
ostream sout;
sout << i;
string s2 = sout.str();
cout << s2 << endl;
```

Nhập xuất kiểu tự định nghĩa

Ta đã biết luồng C++ cung cấp toán tử trích và chèn cho các kiểu dữ liệu dựng sẵn, giúp nhập xuất thuận tiện. Với kiểu tự định nghĩa, ta cũng có thể dùng `>>` và `<<` để nhập xuất hay không? Câu trả lời là có, và rất đơn giản: nạp chồng `>>` và `<<`. Ví dụ:

```

//: C02:Complex.cpp
// Input & Output for User-defined types
#include <iostream>
using namespace std;
class Complex {
public:
    Complex(double r = 0., double i = 0.)
        : re(r), im(i) {}
    double real() const { return re; }
    double imag() const { return im; }
private:
    double re, im;
    friend istream& operator >>(
        istream& is, Complex& c);
    friend ostream& operator <<(
        ostream& os, const Complex& c);
};
istream& operator >>(istream& is, Complex& c) {
    is >> ws;
    if(is.peek() == '(') {
        char chr;
        is >> chr;    // bỏ qua '('
        is >> c.re;
        is >> chr;    // bỏ qua ','
        is >> c.im;
        is >> chr;    // bỏ qua ')'
    }
    else {
        is >> c.re;
        c.im = 0.;
    }
    return is;
}
ostream& operator <<(ostream& os, const Complex& c) {
    return os << '(' << c.re << ',' << c.im << ')';
}
int main() {
    Complex c;
    cin >> c;
    cout << c << endl;
} ///:~

```

Chương trình trên hiện thực một kiểu số phức đơn giản và nạp chồng toán tử trích, chèn. Vì ta không thể sửa mã của lớp luồng, hai toán tử này chỉ có thể được nạp chồng toàn cục. Kiểu trả về là tham chiếu đến luồng, nhờ đó các toán tử trích hoặc chèn có thể dùng liên tiếp. Trong danh sách tham số, tham số đầu tiên phải là tham chiếu đến luồng. Khi nạp chồng toán tử trích, tham số thứ hai cũng phải là tham chiếu, vì khi nhập ta cần thay đổi giá trị biến. Khi nạp chồng toán tử chèn, tham số thứ hai thường là tham chiếu hằng, vừa bảo đảm tốc độ vì tham chiếu truyền như con trỏ, vừa bảo đảm biến không bị sửa.

Khi kết thúc hàm, đừng quên trả về chính luồng để toán tử có thể dùng liên tiếp.

2.5 Kết chương

Có lẽ độc giả đã thấy ở đầu và cuối các chương trình trong bài có vài chú thích lạ. Chúng dùng để trích mã nguồn của từng chương trình từ tập tin văn bản thuần của bài viết. Sau đây là một chương trình thực hiện chức năng đó.

```
//: C02:Extract.cpp
// Extracts codes from documents
#include <iostream>
#include <fstream>
#include <string>
#include <cstdlib>
using namespace std;
void error(const char* msg) {
    cerr << msg << endl;
    exit(1); // similar to Pascal's Halt
}
int main(int argc, char* argv[]) {
    if(argc <= 1)
        error("Document filename missing");
    ifstream fin(argv[1]);
    if(!fin)
        error("Cannot open file");
    string line;
    bool is_in_code = false;
    ofstream fout;
    while(getline(fin, line))
        if(line.find("//:") == 0) {
            if(is_in_code) {
                fout.close();
                error("Begin tag unexpected");
            }
            else {
                is_in_code = true;
                string name = line.substr(line.rfind('.') + 1);
                fout.open(name.c_str());
                if(!fout)
                    error("Cannot create file");
                fout << line << endl;
            }
        }
    else if(line.find("///::~") != string::npos) {
        if(is_in_code) {
            is_in_code = false;
            fout << line << endl;
            fout.close();
        }
    }
}
```

```

    }
    else
        error("End tag unexpected");
}
else if(is_in_code)
    fout << line << endl;
if(is_in_code) {
    fout.close();
    error("End tag expected");
}
fin.close();
} ///:~

```

Chương trình này minh họa chuỗi và luồng đã giới thiệu trong chương. Cách dùng rất đơn giản: sau khi biên dịch, gõ tên chương trình trên dòng lệnh và thêm tên tập tin văn bản thuần của tài liệu làm tham số, chương trình sẽ tự động trích các chương trình nguồn trong bài ra thư mục hiện tại.

Trong chương này, ta đã giới thiệu cơ chế trừu tượng hóa dữ liệu của C++, tức lớp, rồi từ đó giới thiệu một số đặc tính ngôn ngữ xây dựng trên nền lớp. Chúng đều là công cụ mạnh cho việc lập trình.

3 Giới thiệu STL

Điều kiện cần để đọc chương này là hiểu nền tảng lập trình hướng đối tượng của C++ và có một lượng kiến thức thuật toán nhất định. Chương này giới thiệu STL, tức *Standard Template Library*, Thư viện Khuôn mẫu Chuẩn mạnh mẽ do C++ chuẩn cung cấp.

3.1 Khái quát STL

Đôi khi ta thấy một số hàm có cùng chức năng nhưng tham số khác kiểu, khiến ta phải viết nhiều hàm tương tự, rất bất tiện. Để giải quyết vấn đề này, C++ cung cấp phương tiện lập trình tổng quát, trong đó kiểu cũng được xem như tham số. Ví dụ hàm thường dùng để đổi giá trị hai biến:

```

void swap(int& x, int& y) {
    int t = x;
    x = y;
    y = t;
}

```

Hàm trên chỉ đổi được hai biến nguyên. Với kiểu khác, ta buộc phải viết hàm tương tự. Vậy dùng lập trình tổng quát để đơn giản hóa ra sao?

Hàm khuôn mẫu

Trong C++, vấn đề này được giải quyết bằng hàm khuôn mẫu. Khi dùng hàm khuôn mẫu, ta chỉ cần sửa hàm cũ rất ít. Với swap, có thể viết:

```

template <class T> void swap(T& x, T& y) {
    T t = x;
    x = y;
}

```

```

    y = t;
}

```

Trong hàm swap trên, template là tiền tố của hàm khuôn mẫu, biểu thị phần sau là một hàm khuôn mẫu. Nội dung giữa < và > mô tả tham số khuôn mẫu; class T nghĩa là T là một kiểu, không nhất thiết là lớp. Trong định nghĩa hàm, ta dùng T thay cho int ban đầu. Sau khi định nghĩa hàm khuôn mẫu swap, ta có thể dùng nó để đổi giá trị của mọi cặp biến cùng kiểu; trình biên dịch sẽ dựa vào kiểu khác nhau để tạo các phiên bản swap khác nhau, qua đó giảm đáng kể độ phức tạp khi lập trình.

Thông thường trình biên dịch dựa vào kiểu tham số thực khi gọi để đặc biệt hóa hàm khuôn mẫu. Ta cũng có thể tự chỉ định kiểu tham số khi đặc biệt hóa.

```
swap<int>(x, y);
```

Ở đây swap<int>() thực chất giống hàm ban đầu đổi hai biến nguyên.

Lớp khuôn mẫu

Tương tự, ta cũng có thể tổng quát hóa lớp.

```

template <class T, int max> struct c_array {
    typedef T value_type;
    typedef T& reference;
    typedef const T& const_reference;
    T v[max];
    operator T*() { return v; }
    reference operator [](size_t i) { return v[i]; }
    const_reference operator [](size_t i) const { return v[i]; }
    size_t size() const { return max; }
};

```

Đoạn trên định nghĩa lớp khuôn mẫu c_array, có thể thay cho mảng một chiều dựng sẵn của C++. Nó có hai tham số khuôn mẫu: tham số thứ nhất chỉ kiểu phần tử, tham số thứ hai chỉ số phần tử. Nếu muốn định nghĩa một mảng nguyên gồm 100 phần tử, viết:

```
c_array<int, 100> a;
```

Vì đã nạp chồng toán tử chỉ số và toán tử chuyển kiểu, ta có thể dùng lớp này giống mảng thông thường.

Thư viện Khuôn mẫu Chuẩn

STL là thư viện mạnh được xây dựng trên hàm khuôn mẫu và lớp khuôn mẫu. Nhờ hàm khuôn mẫu, ta có thể viết các hàm hỗ trợ gần như mọi kiểu, kể cả kiểu tự định nghĩa, để hiện thực các thuật toán tổng quát thường dùng như đếm, sắp xếp, tìm kiếm. Nhờ lớp khuôn mẫu, ta có thể viết các vật chứa hỗ trợ gần như mọi kiểu, dùng để hiện thực các cấu trúc dữ liệu thường gặp như danh sách liên kết, ngăn xếp, hàng đợi, cây nhị phân cân bằng.

Điều thú vị là khác các thư viện thông thường, các tập tin đầu tạo nên STL chứa toàn bộ mã nguồn của các hàm và lớp. Ta có thể đọc các tập tin đầu này để học cách STL hiện thực các chức năng. Các mục sau sẽ lần lượt giới thiệu các thành phần của STL.

3.2 Bộ lặp

Bộ lặp thực chất là một kiểu con trỏ tổng quát, tức sự trừu tượng hóa của kiểu con trỏ. Dựa trên các thao tác được hỗ trợ, bộ lặp được chia thành năm loại: bộ lặp xuất, bộ lặp nhập, bộ lặp tiến, bộ lặp hai chiều và bộ lặp ngẫu nhiên. Bảng sau liệt kê các thao tác được hỗ trợ.

Loại bộ lặp	Xuất	Nhập	Tiến	Hai chiều	Ngẫu nhiên
Viết tắt	Out	In	For	Bi	Ran
Đọc	không hỗ trợ	$x = *p, p \rightarrow x$	$x = *p, p \rightarrow x$	$x = *p, p \rightarrow x$	$x = *p, p \rightarrow x, p[i]$
Ghi	$*p = x$	không hỗ trợ	$*p = x$	$*p = x$	$*p = x$
Duyệt	++	++	++	++, --	++, --, +, -, +=, -=
So sánh	không hỗ trợ	$== !=$	$== !=$	$== !=$	$== != < > <= >=$

Từ bảng này có thể thấy kiểu con trỏ chính là một kiểu bộ lặp ngẫu nhiên đặc biệt. Với bộ lặp thông thường, các chức năng này được hiện thực bằng nạp chồng toán tử.

Bộ lặp tiến tương thích với bộ lặp xuất và bộ lặp nhập; bộ lặp hai chiều tương thích với bộ lặp tiến; bộ lặp ngẫu nhiên tương thích với bộ lặp hai chiều. Quan hệ tương thích này có tính bắc cầu.

Kiểu cơ sở của bộ lặp được định nghĩa trong tập tin đầu `iterator`. Vì nó là một lớp trừu tượng, định nghĩa trông hơi lạ:

```
template <class Cat, class T, class Dist = ptrdiff_t,
         class Ptr = T*, class Ref = T&>
struct iterator {
    typedef Cat iterator_category;
    typedef T value_type;
    typedef Dist difference_type;
    typedef Ptr pointer;
    typedef Ref reference;
};
```

Lớp `iterator` chỉ cung cấp một giao diện cho kiểu bộ lặp; hiện thực cụ thể sẽ được đặt ở những nơi cần đến.

Bộ lặp không chỉ dùng để truy cập phần tử. Trong toàn bộ STL, nó còn đóng vai trò liên kết thuật toán và vật chứa. Vai trò này sẽ thấy rõ ở các mục sau.

Tập tin đầu `utility` còn định nghĩa lớp khuôn mẫu `pair` khá thường dùng, để biểu diễn bộ đôi:

```
template <class T1, class T2> struct pair {
    typedef T1 first_type;
    typedef T2 second_type;
    T1 first;
    T2 second;
    pair() : first(T1()), second(T2()) { }
    pair(const T1& x, const T2& y) : first(x), second(y) { }
    template<class U, class V> pair(const pair<U, V>& p)
        : first(p.first), second(p.second) { }
};

template <class T1, class T2>
pair<T1, T2> make_pair(const T1& t1, const T2& t2) {
    return pair<T1, T2>(t1, t2);
}
```

Lớp này sẽ xuất hiện nhiều lần khi mô tả thuật toán và vật chứa.

3.3 Thuật toán

Các thuật toán trong STL cung cấp một lượng lớn thao tác cơ bản, phần lớn là thao tác trên dãy. Trước khi giới thiệu thuật toán STL, gọi tắt là thuật toán, ta cần biết vài khái niệm nền tảng liên quan.

Dãy

Đối tượng thao tác của hầu hết thuật toán là dãy. Trong STL, mô tả một dãy cần hai bộ lặp: bộ lặp đầu trở tới phần tử đầu tiên của dãy, bộ lặp thứ hai trở tới vị trí sau phần tử cuối cùng. Nói cách khác, khoảng nửa kín nửa mở do hai bộ lặp làm đầu mút biểu diễn toàn bộ dãy.

Cách định nghĩa này ban đầu có thể lạ, nhưng nó đem lại nhiều tiện lợi. Với dãy $[first, last)$, biểu thức $last - first$ biểu diễn độ dài dãy nếu bộ lặp hỗ trợ phép trừ. Với mảng một chiều s có n phần tử, ta dùng $[s, s + n)$ để mô tả toàn bộ mảng. Khi mô tả vị trí chèn, bộ lặp p có nghĩa là chèn trước phần tử mà p trở tới; như vậy $first$ là chèn vào đầu dãy, $last$ là chèn vào cuối dãy, và mọi vị trí giữa cũng có bộ lặp tương ứng.

Với các vật chứa STL ở mục sau, ta dùng các hàm thành viên do lớp vật chứa cung cấp để tạo bộ lặp, rồi từ đó tạo dãy gồm toàn bộ hoặc một phần phần tử trong vật chứa.

Đối tượng hàm

Một số thuật toán ngoài việc mô tả dãy còn yêu cầu cung cấp đối tượng hàm. Ví dụ:

```
template<class In, class Op>
    Op for_each(In first, In last, Op f);
```

Hàm này thực hiện $f(x)$ một lần với mỗi phần tử x trong $[first, last)$. Tất nhiên ta có thể dùng tên hàm làm tham số f , nhưng với một số thao tác, chẳng hạn tính tổng, có thể cần vài biến toàn cục. Nếu định nghĩa các biến này toàn cục, ta phá vỡ nguyên tắc đóng gói. Cách giải quyết là dùng đối tượng hàm. Với phép tính tổng, có thể viết lớp sau:

```
template<class T> class Sum {
    T res;
public:
    Sum(T i = T()) : res(i) { }
    void operator()(const T& x) { res += x; }
    T result() const { return res; }
};
```

Khi đó có thể tính tổng một dãy như sau:

```
Sum<int> sum;
for_each(s, s + n, sum(0));
cout << sum.result() << endl;
```

Thực ra STL đã cung cấp thuật toán tính tổng; cách trên chỉ để minh họa cách dùng đối tượng hàm.

Đối tượng hàm, kể cả hàm thường, có kiểu trả về là `bool` được gọi là vị từ, thường dùng để kiểm tra một quan hệ nào đó có thành lập hay không.

Trong tập tin đầu `functional`, STL cung cấp một số đối tượng hàm cho các phép toán thường dùng, đều là lớp khuôn mẫu.

Tên lớp	Loại	Tác dụng
equal_to	hai ngôi	<code>arg1 == arg2</code>
not_equal_to	hai ngôi	<code>arg1 != arg2</code>
greater	hai ngôi	<code>arg1 > arg2</code>
less	hai ngôi	<code>arg1 < arg2</code>
greater_equal	hai ngôi	<code>arg1 >= arg2</code>
less_equal	hai ngôi	<code>arg1 <= arg2</code>
logical_and	hai ngôi	<code>arg1 && arg2</code>
logical_or	hai ngôi	<code>arg1 arg2</code>
logical_not	một ngôi	<code>!arg</code>
plus	hai ngôi	<code>arg1 + arg2</code>
minus	hai ngôi	<code>arg1 - arg2</code>
multiplies	hai ngôi	<code>arg1 * arg2</code>
divides	hai ngôi	<code>arg1 / arg2</code>
modulus	hai ngôi	<code>arg1 % arg2</code>
negate	một ngôi	<code>-arg</code>

Khái quát các thuật toán

STL cung cấp hơn 60 thuật toán. Chúng đều là hàm khuôn mẫu và thường có hai phiên bản nạp chồng: một phiên bản dùng toán tử mặc định, thường là bằng hoặc nhỏ hơn tùy loại thuật toán; phiên bản còn lại cho phép tự chỉ định đối tượng hàm thay cho toán tử mặc định.

Bảng sau liệt kê các thuật toán STL. Trừ bốn thuật toán cuối được định nghĩa trong `numeric`, các thuật toán còn lại nằm trong `algorithm`.

Tên thuật toán	Tác dụng
<code>for_each()</code>	Thực hiện một thao tác với mỗi phần tử của dãy
<code>find()</code>	Tìm vị trí xuất hiện đầu tiên của một giá trị trong dãy
<code>find_if()</code>	Tìm phần tử đầu tiên thỏa một vị từ
<code>find_first_of()</code>	Tìm vị trí đầu tiên trong một dãy chứa phần tử bất kỳ của dãy khác
<code>adjacent_find()</code>	Tìm cặp phần tử kề nhau đầu tiên bằng nhau
<code>count()</code>	Đếm số lần một giá trị xuất hiện trong dãy
<code>count_if()</code>	Đếm số phần tử thỏa một vị từ
<code>mismatch()</code>	Tìm cặp phần tử khác nhau đầu tiên của hai dãy
<code>equal()</code>	Kiểm tra hai dãy có hoàn toàn bằng nhau không
<code>search()</code>	Tìm vị trí xuất hiện đầu tiên của một dãy con
<code>find_end()</code>	Tìm vị trí xuất hiện cuối cùng của một dãy con
<code>search_n()</code>	Tìm vị trí một giá trị xuất hiện liên tiếp n lần
<code>transform()</code>	Thực hiện thao tác trên từng phần tử và tạo dãy xuất tương ứng
<code>copy()</code>	Sao chép dãy sang dãy khác từ trước ra sau
<code>copy_backward()</code>	Sao chép dãy sang dãy khác từ sau ra trước
<code>swap()</code>	Đổi giá trị hai phần tử
<code>iter_swap()</code>	Đổi giá trị hai phần tử mà hai bộ lặp trở tới
<code>swap_ranges()</code>	Đổi phần tử giữa hai dãy
<code>replace()</code>	Thay một giá trị trong dãy bằng giá trị khác
<code>replace_if()</code>	Thay các phần tử thỏa một vị từ bằng giá trị khác

Tên thuật toán	Tác dụng
replace_copy()	Như replace() nhưng ghi kết quả sang dãy khác
replace_copy_if()	Như replace_if() nhưng ghi kết quả sang dãy khác
fill()	Gán mọi phần tử trong dãy bằng một giá trị
fill_n()	Như fill() nhưng dãy được mô tả bằng bộ lặp đầu và độ dài
generate()	Gán mọi phần tử bằng giá trị trả về của bộ sinh
generate_n()	Như generate() nhưng dãy được mô tả bằng bộ lặp đầu và độ dài
remove()	Loại khỏi dãy các phần tử bằng một giá trị
remove_if()	Loại khỏi dãy các phần tử thỏa một vị từ
remove_copy()	Như remove() nhưng ghi kết quả sang dãy khác
remove_copy_if()	Như remove_if() nhưng ghi kết quả sang dãy khác
unique()	Loại các phần tử bằng nhau kề nhau bị thừa
unique_copy()	Như unique() nhưng ghi kết quả sang dãy khác
reverse()	Đảo ngược dãy
reverse_copy()	Như reverse() nhưng ghi kết quả sang dãy khác
rotate()	Xoay dãy
rotate_copy()	Như rotate() nhưng ghi kết quả sang dãy khác
random_shuffle()	Xáo trộn ngẫu nhiên các phần tử trong dãy
sort()	Sắp xếp các phần tử trong dãy
stable_sort()	Như sort() nhưng bảo đảm tính ổn định
partial_sort()	Sắp xếp phần đầu của dãy
partial_sort_copy()	Như partial_sort() nhưng ghi kết quả sang dãy khác
nth_element()	Đưa phần tử nhỏ thứ n về đúng vị trí của nó sau khi sắp xếp
lower_bound()	Tìm nhị phân trong dãy có thứ tự, trả về vị trí xuất hiện đầu tiên
upper_bound()	Tìm nhị phân trong dãy có thứ tự, trả về vị trí sau lần xuất hiện cuối
equal_range()	Tìm nhị phân và trả về dãy con gồm mọi phần tử bằng giá trị cần tìm
binary_search()	Kiểm tra một giá trị có xuất hiện trong dãy có thứ tự không
merge()	Trộn hai dãy có thứ tự
inplace_merge()	Trộn hai dãy có thứ tự liên tiếp
partition()	Sắp lại dãy để phần tử thỏa vị từ đứng trước
stable_partition()	Như partition() nhưng bảo đảm tính ổn định
includes()	Kiểm tra một dãy có là dãy con, không cần liên tiếp, của dãy khác không
set_union()	Hợp hai dãy có thứ tự theo nghĩa tập hợp
set_intersection()	Giao hai dãy có thứ tự theo nghĩa tập hợp
set_difference()	Hiệu hai dãy có thứ tự theo nghĩa tập hợp
set_symmetric_difference()	Hiệu đối xứng hai dãy có thứ tự
make_heap()	Điều chỉnh một dãy thành heap
push_heap()	Chèn một phần tử vào heap
pop_heap()	Lấy phần tử đỉnh heap ra
sort_heap()	Điều chỉnh heap thành dãy có thứ tự
min()	Trả về giá trị nhỏ hơn trong hai giá trị
max()	Trả về giá trị lớn hơn trong hai giá trị

Tên thuật toán	Tác dụng
<code>min_element()</code>	Trả về vị trí phần tử nhỏ nhất trong dãy
<code>max_element()</code>	Trả về vị trí phần tử lớn nhất trong dãy
<code>lexicographical_compare()</code>	So sánh hai dãy theo thứ tự từ điển
<code>next_permutation()</code>	Trả về hoán vị kế tiếp theo thứ tự từ điển
<code>prev_permutation()</code>	Trả về hoán vị trước theo thứ tự từ điển
<code>accumulate()</code>	Tính tổng các phần tử của dãy
<code>inner_product()</code>	Tính tích vô hướng của hai dãy
<code>partial_sum()</code>	Tính dãy tổng từng phần
<code>adjacent_difference()</code>	Tính dãy hiệu của các phần tử kề nhau

Tính ổn định trong sắp xếp nghĩa là trong dãy sau khi sắp xếp, các phần tử có khóa bằng nhau giữ nguyên thứ tự tương đối như trong dãy ban đầu.

Bảng trên chỉ giới thiệu chức năng cơ bản. Sau đây là vài thuật toán thường dùng. Trong thi tin học, thuật toán được dùng nhiều nhất là sắp xếp. Hàm `sort()` của STL làm việc này rất thuận tiện.

```
template<class Ran> void sort(Ran first, Ran last);
template<class Ran, class Cmp>
    void sort(Ran first, Ran last, Cmp cmp);
```

Với kiểu đã định nghĩa toán tử nhỏ hơn, chỉ cần viết `sort(s, s + n)` là sắp xếp xong. Nếu chưa định nghĩa toán tử nhỏ hơn hoặc muốn dùng quan hệ khác để sắp xếp, dùng phiên bản nạp chồng thứ hai. Khi đó ngoài dãy cần sắp xếp, ta đưa thêm một vị từ làm căn cứ so sánh. Ví dụ `sort(s, s + n, greater<int>())` sẽ sắp xếp giảm dần.

`sort()` thường được hiện thực bằng quicksort, thời gian trung bình $O(n \log n)$. Trường hợp xấu nhất là $O(n^2)$, nhưng xác suất gặp thường rất nhỏ.

Tìm nhị phân cũng là thuật toán rất thường dùng trong thi tin học. STL có bốn thuật toán liên quan:

```
template<class For, class T> bool
    binary_search(For first, For last, const T& val);
template<class For, class T, class Cmp> bool
    binary_search(For first, For last, const T& val, Cmp cmp);
template<class For, class T> For
    lower_bound(For first, For last, const T& val);
template<class For, class T, class Cmp> For
    lower_bound(For first, For last, const T& val, Cmp cmp);
template<class For, class T> For
    upper_bound(For first, For last, const T& val);
template<class For, class T, class Cmp> For
    upper_bound(For first, For last, const T& val, Cmp cmp);
template<class For, class T> pair<For, For>
    equal_range(For first, For last, const T& val);
template<class For, class T, class Cmp> pair<For, For>
    equal_range(For first, For last, const T& val, Cmp cmp);
```

Nhìn tên thì thuật toán tìm nhị phân phải là `binary_search()`, nhưng thực tế hàm này không quá hữu dụng vì chỉ trả về một giá trị có xuất hiện trong dãy hay không. Ba hàm sau mới thường hữu ích hơn.

Nếu giá trị cần tìm xuất hiện trong dãy, `lower_bound()` trả về vị trí phần tử đầu tiên bằng giá trị đó, còn `upper_bound()` trả về vị trí ngay sau phần tử cuối cùng bằng giá trị đó. Cách định nghĩa này phù hợp với định nghĩa dãy trong STL. Gọi giá trị trả về của `lower_bound()` là l và của `upper_bound()` là u , thì dãy $[l, u)$ chính là dãy con gồm mọi phần tử bằng giá trị cần tìm. Nếu giá trị cần tìm không xuất hiện, hai hàm trả về cùng một vị trí, sao cho nếu chèn giá trị cần tìm vào đó thì dãy vẫn giữ thứ tự. `equal_range()` là phiên bản gộp của `lower_bound()` và `upper_bound()`. Cách dùng chi tiết các thuật toán khác nên xem tài liệu.

Ví dụ: dãy con tăng dài nhất

Mô tả bài toán. Cho dãy số nguyên độ dài n , $A = \{a_1, a_2, \dots, a_n\}$. Hãy tìm số nguyên lớn nhất m sao cho tồn tại một dãy chỉ số $P = \{p_1, p_2, \dots, p_m\}$, thỏa $1 \leq p_1 < p_2 < \dots < p_m \leq n$ và $a_{p_1} < a_{p_2} < \dots < a_{p_m}$.

Dữ liệu vào. Tập tin `LMIS.in`. Dòng thứ nhất là số nguyên dương $n \leq 30000$, độ dài dãy ban đầu. Dòng thứ hai là n số nguyên dương $a_i \leq 1000000000$.

Dữ liệu ra. Tập tin `LMIS.out`. Một dòng chứa số nguyên m , độ dài của dãy con tăng dài nhất.

Input

10

3 1 4 1 5 9 2 6 5 3

Output

4

Đây là một bài quy hoạch động điển hình. Gọi f_i là độ dài dãy con tăng dài nhất kết thúc tại phần tử thứ i của dãy ban đầu. Để tiện, đặt $a_0 = -\infty, f_0 = 0$. Phương trình chuyển trạng thái:

$$f_i = \max_{0 \leq j < i, a_j < a_i} \{f_j + 1\}.$$

Kết quả cần tìm là giá trị lớn nhất trong các f_i . Chương trình hiện thực thuật toán này như sau; vì chỉ số mảng C++ bắt đầu từ 0, chỉ số được điều chỉnh đôi chút.

```
//: C03:LMIS1.cpp
// Longest Monotonically Increasing Subsequence
// Original Algorithm O(n^2)
#include <fstream>
#include <algorithm>
using namespace std;
ifstream fin("LMIS.in");
ofstream fout("LMIS.out");
const int max_n = 30000;
int n;
int a[max_n];
int f[max_n];
int main() {
    fin >> n;
    for(int i = 0; i < n; i++)
```

```

        fin >> a[i];
    for(int i = 0; i < n; i++) {
        f[i] = 1;
        for(int j = 0; j < i; j++)
            if(a[j] < a[i])
                f[i] = max(f[i], f[j] + 1);
    }
    fout << *max_element(f, f + n) << endl;
} ///:~

```

Thuật toán này có thời gian $O(n^2)$, không đáp ứng dữ liệu. Để cải tiến, ta dùng một mảng phụ. Gọi g_i là giá trị nhỏ nhất của phần tử cuối trong tất cả dãy con tăng độ dài i đã thấy đến hiện tại. Dễ biết $g_{i-1} \leq g_i$ với $1 \leq i \leq n$, có thể chứng minh bằng phản chứng; tức $\{g_i\}$ là dãy không giảm. Khi đã biết $\{g_i\}$ đến phần tử thứ $i-1$, f_i bằng vị trí của phần tử đầu tiên trong $\{g_i\}$ lớn hơn hoặc bằng a_i , cũng chính là vị trí do `lower_bound()` trả về. Sau đó cập nhật $g_{f_i} = a_i$. Ban đầu đặt $g_i = \infty$ với $1 \leq i \leq n$. Mỗi lần tìm vị trí mất $O(\log n)$, cập nhật mất $O(1)$, nên tổng thời gian là $O(n \log n)$.

```

//: C03:LMIS2.cpp
// Longest Monotonically Increasing Subsequence
// Improved Algorithm  $O(n \log n)$ 
#include <fstream>
#include <algorithm>
using namespace std;
ifstream fin("LMIS.in");
ofstream fout("LMIS.out");
const int infinity = 2000000000;
const int max_n = 30000;
int n;
int a[max_n];
int f[max_n];
int g[max_n];
int main() {
    fin >> n;
    for(int i = 0; i < n; i++)
        fin >> a[i];
    fill(g, g + n, infinity);
    for(int i = 0; i < n; i++) {
        int j = lower_bound(g, g + n, a[i]) - g;
        f[i] = j + 1;
        g[j] = a[i];
    }
    fout << *max_element(f, f + n) << endl;
} ///:~

```

So sánh hai chương trình cho thấy nhờ STL, chương trình cải tiến thậm chí còn đơn giản hơn chương trình ban đầu, thể hiện rõ ưu điểm giảm độ phức tạp khi lập trình.

3.4 Vật chứa

Thuật toán chỉ phát huy tác dụng khi tác động lên cấu trúc dữ liệu. Ngoài nhiều thuật toán, STL còn cung cấp một số cấu trúc dữ liệu thường dùng. Vì chúng đều dùng để chứa một nhóm đối tượng cùng loại, chúng được gọi là vật chứa. Bảng sau liệt kê các vật chứa STL phổ biến.

Tên	Mô tả	Loại	Tập tin đầu	Bộ lặp
vector	vector	vật chứa thường	vector	ngẫu nhiên
deque	hàng đợi hai đầu	vật chứa thường	deque	ngẫu nhiên
list	danh sách liên kết	vật chứa thường	list	hai chiều
stack	ngăn xếp	bộ thích nghi	stack	không cung cấp
queue	hàng đợi	bộ thích nghi	queue	không cung cấp
priority_queue	hàng đợi ưu tiên	bộ thích nghi	queue	không cung cấp
set	tập hợp	vật chứa kết hợp	set	hai chiều
multiset	đa tập	vật chứa kết hợp	set	hai chiều
map	ánh xạ	vật chứa kết hợp	map	hai chiều
multimap	đa ánh xạ	vật chứa kết hợp	map	hai chiều

Bộ thích nghi vật chứa chỉ cung cấp một giao diện hàm, không cung cấp bộ lặp, và bên trong chứa một vật chứa thường để thực hiện lưu trữ phần tử. Vật chứa kết hợp thường có phần tử gồm khóa và giá trị được ánh xạ, cho phép tìm nhanh theo khóa.

Mọi vật chứa đều được hiện thực bằng lớp khuôn mẫu, nên khi dùng phải chỉ định kiểu phần tử. Điều này có thể làm tên kiểu dài và viết bất tiện; khi cần có thể dùng typedef để đơn giản hóa.

```
typedef vector<string> str_vec;
```

Vật chứa STL tự động cấp phát không gian cho phần tử. Ví dụ vector sẽ giữ trước một phần không gian; khi chèn quá nhiều phần tử, nó tự cấp phát một vùng lớn hơn và sao chép phần tử cũ sang. Khi đối tượng vật chứa bị hủy, không gian của phần tử cũng tự được thu hồi.

Với các vật chứa này, STL cung cấp hệ thống kiểu thành viên và giao diện hàm cơ bản tương đối giống nhau, dù từng vật chứa có khác biệt.

Tên	Loại	Mô tả ngắn
value_type	kiểu thành viên	Kiểu phần tử
iterator	kiểu thành viên	Kiểu bộ lặp, giống value_type*
const_iterator	kiểu thành viên	Kiểu bộ lặp hằng, giống const value_type*
reverse_iterator	kiểu thành viên	Kiểu bộ lặp ngược
const_reverse_iterator	kiểu thành viên	Kiểu bộ lặp ngược hằng
reference	kiểu thành viên	Tham chiếu phần tử, bằng value_type&
const_reference	kiểu thành viên	Tham chiếu hằng phần tử
key_type	kiểu thành viên	Kiểu khóa, chỉ dùng cho vật chứa kết hợp
mapped_type	kiểu thành viên	Kiểu được ánh xạ, chỉ dùng cho vật chứa kết hợp
begin()	thao tác bộ lặp	Trả về bộ lặp trở tới phần tử đầu
end()	thao tác bộ lặp	Trả về bộ lặp trở tới vị trí sau phần tử cuối
rbegin()	thao tác bộ lặp	Trả về bộ lặp ngược trở tới phần tử cuối
rend()	thao tác bộ lặp	Trả về bộ lặp ngược trở tới trước phần tử đầu
front()	thao tác phần tử	Phần tử đầu

Tên	Loại	Mô tả ngắn
<code>back()</code>	thao tác phần tử	Phần tử cuối
<code>operator []()</code>	thao tác phần tử	Toán tử chỉ số
<code>push_back()</code>	ngăn xếp/hàng đợi	Thêm một phần tử vào cuối
<code>pop_back()</code>	ngăn xếp/hàng đợi	Xóa phần tử cuối
<code>push_front()</code>	ngăn xếp/hàng đợi	Thêm một phần tử vào đầu, vector không hỗ trợ
<code>pop_front()</code>	ngăn xếp/hàng đợi	Xóa phần tử đầu, vector không hỗ trợ
<code>insert()</code>	thao tác danh sách	Chèn một số phần tử
<code>erase()</code>	thao tác danh sách	Xóa một số phần tử
<code>clear()</code>	thao tác danh sách	Xóa sạch vật chứa
<code>size()</code>	thao tác khác	Trả về số phần tử
<code>empty()</code>	thao tác khác	Kiểm tra vật chứa rỗng
<code>resize()</code>	thao tác khác	Điều chỉnh số phần tử
<code>find()</code>	vật chứa kết hợp	Tìm một khóa
<code>lower_bound()</code>	vật chứa kết hợp	Tìm khóa, trả về vị trí đầu phù hợp
<code>upper_bound()</code>	vật chứa kết hợp	Tìm khóa, trả về vị trí sau vị trí cuối phù hợp
<code>equal_range()</code>	vật chứa kết hợp	Tìm khóa, trả về khoảng phù hợp

Sau đây là vài vật chứa thường dùng.

`vector` là mảng một chiều thông minh có thể tự điều chỉnh không gian lưu trữ. `vector` còn tối ưu cách lưu `vector<bool>`, dùng mỗi bit để lưu một giá trị `bool`. Tuy nhiên trong thi tin học, quy mô dữ liệu thường đã biết trước, hơn nữa việc điều chỉnh không gian của `vector` phải di chuyển dữ liệu và tốn khá nhiều thời gian, nên `vector` không phải lúc nào cũng thực dụng nhất.

`deque` được hiện thực bằng mảng con trỏ và chứa một lượng không gian ở cả hai đầu, nên thao tác chèn/xóa ở đầu và cuối rất nhanh. `deque` thường dùng làm vật chứa bên trong cho các bộ thích nghi như `stack` và `queue`.

`list` thường được hiện thực bằng danh sách liên kết hai chiều, giúp ta tránh phải viết nhiều thao tác danh sách cơ bản lặp lại và giảm độ phức tạp khi lập trình.

`stack`, `queue` và `priority_queue` đều là bộ thích nghi vật chứa. Vì giao diện ít hàm, công dụng của `stack` và `queue` có phần bị giới hạn, nhưng đủ cho các tình huống thông thường. `priority_queue` là hàng đợi ưu tiên hiện thực bằng heap nhị phân. Khác hàng đợi thường, khi lấy phần tử, hàng đợi ưu tiên luôn lấy phần tử có độ ưu tiên cao nhất trước; với cách so sánh mặc định của C++ chuẩn, phần tử lớn nhất đứng đầu. `priority_queue` rất hữu ích trong bài mô phỏng, vì mấu chốt của mô phỏng là xử lý đúng thứ tự sự kiện. Ta có thể xem các sự kiện trong đề là đối tượng, gán độ ưu tiên theo quy tắc đề, nạp chồng toán tử so sánh, rồi lần lượt đẩy vào hàng đợi ưu tiên và lấy ra xử lý. Tuy nhiên do giới hạn giao diện, hàng đợi ưu tiên khó xử lý tình huống phần tử trong hàng đợi cần thay đổi, chẳng hạn trong thuật toán Dijkstra hoặc Prim.

Các vật chứa kết hợp của STL rất mạnh. Bốn vật chứa này đều được hiện thực bằng cây nhị phân cân bằng, thường là cây đỏ-đen. Phần tử trong vật chứa có thứ tự, và các thao tác chèn, xóa, tìm đều hoàn thành trong $O(\log n)$. Trong `set` và `multiset`, bản thân phần tử trực tiếp tham gia sắp xếp, hay nói cách khác phần tử chính là khóa. Trong `map` và `multimap`, chỉ khóa tham gia sắp xếp; giá trị được ánh xạ đi kèm như dữ liệu phụ và có thể sửa. Ngoài ra, `set` và `map` không cho phép khóa bằng nhau, còn `multiset` và `multimap` cho phép khóa bằng nhau.

Các vật chứa kết hợp đều có thể tìm phần tử theo khóa. Các hàm thành viên tìm kiếm gồm `find()`, `lower_bound()`, `upper_bound()` và `equal_range()`. Khi không tìm thấy khóa yêu cầu, `find()` trả về giá trị của `end()` để biểu thị thất bại; ba hàm còn lại có giá trị trả về giống các hàm cùng tên trong

algorithm. Vì trong map khóa và giá trị được ánh xạ tương ứng một-một, lớp map nạp chồng toán tử chỉ số, làm việc truy cập phần tử thuận tiện hơn.

```
map<string, int> week;
week["Sunday"] = 0;
week["Monday"] = 1;
week["Tuesday"] = 2;
week["Wednesday"] = 3;
week["Thursday"] = 4;
week["Friday"] = 5;
week["Saturday"] = 6;
cout << week["Thursday"] << endl;
```

Khi dùng toán tử chỉ số của map, nếu phần tử ứng với khóa trong ngoặc vuông tồn tại, tham chiếu đến giá trị được ánh xạ của phần tử đó được trả về. Nếu phần tử ứng với khóa đó chưa tồn tại, một phần tử mới có khóa ấy được chèn vào, giá trị được ánh xạ được khởi tạo bằng giá trị mặc định của kiểu, và tham chiếu đến giá trị được ánh xạ được trả về.

Ví dụ: thanh toán hóa đơn

Mô tả bài toán. Bill gần đây gặp một việc phiền. Mỗi buổi sáng anh nhận một số hóa đơn, có thể không nhận hóa đơn nào; mỗi hóa đơn có một mệnh giá. Buổi chiều, anh chọn hai hóa đơn chưa thanh toán có mệnh giá lớn nhất và nhỏ nhất rồi trả chúng. Các hóa đơn chưa trả được giữ lại đến ngày sau. Biết số lượng và mệnh giá hóa đơn Bill nhận mỗi ngày, hãy giúp anh đưa ra thứ tự thanh toán.

Dữ liệu vào. Dòng thứ nhất là số nguyên dương $N \leq 30000$, tổng số ngày. Từ dòng thứ hai đến dòng $N + 1$, mỗi dòng mô tả một ngày: trước hết là số nguyên không âm M , số hóa đơn nhận trong ngày, sau đó là M số nguyên dương nhỏ hơn 1000000000, mệnh giá các hóa đơn. Dữ liệu bảo đảm mỗi ngày đều có thể thanh toán hai hóa đơn, và đến ngày cuối mọi hóa đơn đều được thanh toán.

Dữ liệu ra. Gồm N dòng, mỗi dòng hai số nguyên cách nhau bởi khoảng trắng, lần lượt là mệnh giá nhỏ nhất và lớn nhất được thanh toán trong ngày.

Input

```
4
3 3 6 5
2 8 2
3 7 1 7
0
```

Output

```
3 6
2 8
1 7
5 7
```

Từ giới hạn dữ liệu có thể thấy thuật toán cần tốt hơn $O(n^2)$, và $O(n \log n)$ chấp nhận được. Có nhiều thuật toán $O(n \log n)$. Chẳng hạn có thể tạo hai heap, một heap lớn nhất và một heap nhỏ nhất, rồi thiết

lập ánh xạ giữa các phần tử tương ứng. Khi nhận hóa đơn, đẩy mệnh giá vào cả hai heap và điều chỉnh vị trí; khi thanh toán, lấy phần tử đầu của mỗi heap và điều chỉnh. Vì vị trí của phần tử trong heap cần thay đổi sau khi đưa vào, khó dùng `priority_queue` của STL để hiện thực; nếu chọn thuật toán này, phải tự viết mã heap.

Một thuật toán $O(n \log n)$ khác là dùng cây nhị phân cân bằng. Tự viết cây cân bằng còn phức tạp hơn cách trên. Nhưng tập hợp của STL được hiện thực bằng cây đỏ-đen, nên nhờ nó chương trình rất đơn giản. Vì mệnh giá hóa đơn có thể trùng nhau, phải dùng `multiset`.

```
//: C03:Bills.cpp
// Pay 2 Bills Every Day
#include <iostream>
#include <set>
using namespace std;
int main() {
    int n;
    cin >> n;
    multiset<int> bills;
    while(n-- > 0) {
        int m;
        cin >> m;
        while(m-- > 0) {
            int a;
            cin >> a;
            bills.insert(a);
        }
        cout << *bills.begin() << ' ' << *--bills.end() << endl;
        bills.erase(bills.begin());
        bills.erase(--bills.end());
    }
} ///:~
```

Ví dụ này một lần nữa thể hiện sức mạnh của STL trong việc giảm độ phức tạp khi lập trình.

Ngoài các vật chứa STL nói trên, C++ chuẩn còn cung cấp `bitset`, tập bit tương tự `set` của Pascal, và `valarray`, mảng số học có thể thực hiện phép toán trên một nhóm dữ liệu cùng kiểu và có thể dùng để hiện thực ma trận. Nhiều trình biên dịch C++ dùng bản STL do SGI hiện thực, còn cung cấp `slist`, danh sách liên kết đơn tiết kiệm không gian, `hash_set`, `hash_multiset`, `hash_map`, `hash_multimap`, các tập hợp và ánh xạ hiện thực bằng bảng băm, cùng `rope`, một cấu trúc dữ liệu chuỗi hỗ trợ chèn và xóa trong thời gian logarit. Các vật chứa này đều thực dụng; độc giả quan tâm có thể xem thêm <http://www.sgi.com/tech/stl>.

3.5 Kết chương

Ba mục trước đã trình bày ba thành phần quan trọng của STL: bộ lập, thuật toán và vật chứa. Nhưng ba phần này không độc lập; chỉ khi kết hợp hữu cơ với nhau, STL mới phát huy đúng tác dụng. Khi vận dụng tổng hợp, cần chú ý một số điểm.

Trước hết, ưu tiên dùng thuật toán do chính vật chứa cung cấp. Một số lớp vật chứa có các hàm thành viên với chức năng gần giống thuật toán tổng quát, và nên ưu tiên dùng chúng. Thuật toán tổng quát được thiết kế cho dãy nói chung, phải cân nhắc tính phổ quát nên khó tối ưu sâu; còn hàm thành viên của lớp

vật chứa có thể dựa trên đặc điểm riêng của vật chứa để tối ưu hoặc làm những việc thuật toán tổng quát không làm được. Ví dụ lớp `list` có các hàm thành viên `remove()`, `unique()`, `merge()`, `reverse()`, `sort()`, nên ưu tiên dùng. Thuật toán tổng quát `sort()` không thể sắp xếp phần tử trong `list` vì `list` chỉ cung cấp bộ lặp hai chiều, còn `sort()` cần bộ lặp ngẫu nhiên.

Thứ hai, khi kết hợp vật chứa và thuật toán cần chú ý dãy xuất. Nhiều thuật toán yêu cầu chỉ định dãy xuất, thường biểu diễn bằng bộ lặp trở tới đầu dãy xuất. Trong thuật toán, bộ lặp này liên tục được tăng để xuất toàn bộ dãy. Vì bộ lặp thông thường không kiểm tra vượt biên, nếu không bảo đảm vật chứa đã có đủ phần tử để lưu dãy xuất, không nên dùng trực tiếp bộ lặp của vật chứa để chỉ định dãy xuất. Cách đúng là dùng bộ lặp chèn trong `iterator`.

Có ba loại bộ lặp chèn: bộ lặp chèn đầu, bộ lặp chèn cuối và bộ lặp chèn thông thường. Với vật chứa `c`, chúng lần lượt được tạo bởi `front_inserter(c)`, `back_inserter(c)` và `inserter(c, i)`, trong đó `i` là một bộ lặp của `c`. Chúng đều là bộ lặp xuất, và khi tăng sẽ thực hiện thao tác chèn tương ứng: `push_front()`, `push_back()` hoặc `insert()`.

```
unique_copy(v.begin(), v.end(), back_inserter(v2));
set_union(s1.begin(), s1.end(), s2.begin(), s2.end(),
         inserter(s, s.begin()));
```

Câu lệnh đầu xóa các phần tử lặp kề nhau trong vector `v` và thêm kết quả vào cuối vector `v2`. Câu lệnh thứ hai lấy hợp của hai tập hợp `s1` và `s2` rồi thêm kết quả vào tập hợp `s`; ở đây `s.begin()` chỉ để đủ số tham số, vì phần tử trong `set` vẫn được sắp theo thứ tự, không phụ thuộc vị trí chèn.

Qua chương này ta đã có hiểu biết nhất định về STL trong C++. STL có nhiều ưu điểm: giảm độ phức tạp khi lập trình và nâng cao xác suất viết đúng mã. Tuy nhiên cái gì có lợi cũng có mặt trái. Vì STL dùng cơ chế lập trình tổng quát như lớp khuôn mẫu và hàm khuôn mẫu, chỉ cần chương trình có lỗi nhỏ cũng có thể sinh ra rất nhiều thông báo lỗi kỳ lạ lúc biên dịch, và vị trí báo lỗi thường nằm trong tập tin đầu của STL, vì lỗi khuôn mẫu thường chỉ lộ ra khi đặc biệt hóa. Lại vì STL dùng đóng gói lớp, việc gỡ lỗi động cũng bất tiện hơn, do ta khó xem trực tiếp các phần tử bên trong vật chứa.

Một số đề nghị khi dùng STL trong thi tin học:

1. Dùng nhiều thuật toán STL để giảm độ phức tạp khi lập trình và tăng độ đúng đắn.
2. Trong điều kiện tương đương, ưu tiên dùng mảng dựng sẵn của C++ thay vì vật chứa STL, để tiện quan sát khi gỡ lỗi.
3. Cố gắng dùng kiểm tra tĩnh nhiều hơn kiểm tra động.
4. Khi gỡ lỗi động, dùng cách xuất ra màn hình để xem các nội dung mà trình gỡ lỗi không xem được.

Tổng kết

Quá trình học ngôn ngữ lập trình có nhiều điểm rất giống quá trình học ngôn ngữ tự nhiên.

Chương thứ nhất trình bày quá trình chuyển từ Pascal sang C++. Khi học, ta phải tận dụng đầy đủ các điểm giống nhau giữa hai ngôn ngữ và nhận rõ các điểm khác biệt, nhờ đó tăng tốc độ chuyển đổi. Điều này giống như học ngoại ngữ trên nền tiếng mẹ đẻ: tận dụng tốt điểm giống giữa các ngôn ngữ làm việc học ngoại ngữ đơn giản hơn; nếu không xử lý tốt các điểm khác, việc học sẽ gặp khó khăn. Muốn học ngoại ngữ tốt cần luyện tập nhiều; tương tự, chỉ khi liên tục dùng một ngôn ngữ lập trình để viết chương trình, ta mới thật sự nâng cao mức độ nắm vững ngôn ngữ đó.

Chương thứ hai và thứ ba đi sâu hơn, trình bày một số nội dung nâng cao của C++. Điều này giống như những tầng văn hóa riêng trong ngoại ngữ; chỉ khi hiểu những tầng ấy, ta mới thật sự nắm một ngôn

ngữ. Ngôn ngữ lập trình cũng vậy. Nội dung chương thứ hai và thứ ba giúp ta dùng C++ tốt hơn, đặc biệt là STL trong chương thứ ba, vốn giảm đáng kể độ phức tạp khi lập trình, nâng cao độ đúng của mã và rất hữu ích cho thi tin học.

Lập trình trong thi tin học lại giống viết văn. Nền tảng của viết văn là nắm tốt một ngôn ngữ tự nhiên; nền tảng của lập trình là nắm tốt một ngôn ngữ lập trình. Nhưng muốn viết bài văn hay thì chỉ nắm ngôn ngữ tự nhiên là chưa đủ, còn cần ý tưởng tinh tế và cách diễn đạt đẹp. Lập trình trong thi tin học cũng vậy: nắm vững ngôn ngữ lập trình rất quan trọng, nhưng cốt lõi của lập trình là thiết kế thuật toán. Chỉ khi kết hợp hữu cơ hai mặt này, ta mới có thể vượt qua các bài toán trong thi tin học và đạt kết quả tốt.

Tài liệu tham khảo

1. *The C++ Programming Language*, Special Edition.
2. *Thinking in C++*, 2nd Edition.
3. *Introduction to Algorithms*, 2nd Edition.